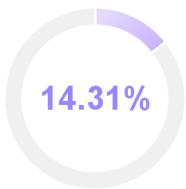


NO. 79ff2039b7m18x1f | 2026-05-16 20:40:36

- 题目：网络流量检测管理系统的设计与开发
- 作者：郭子谊
- 检测所属单位： -

📄 论文字符数：63265 📄 论文页数：92 📊 表格数量：16 🖼️ 图片数量：27

检测结果



14.31%

疑似AIGC生成占比

85.69%

人工撰写占比

结果分布

序号	章节	疑似AIGC生成 文字/章节总字数	疑似AIGC生成 章节占比	人工撰写占比
1	摘要	864/906	95.0%	5.0%
2	Abstract	422/435	97.0%	3.0%
3	1 绪论	2123/5532	38.0%	62.0%
4	2 系统相关技术	293/7344	3.0%	97.0%
5	3 系统需求分析	826/6184	13.0%	87.0%
6	4 系统总体设计	0/5479	0.0%	100.0%
7	5 系统详细设计与实现	1267/13667	9.0%	91.0%
8	6 系统测试与结果分析	425/5367	7.0%	93.0%

*注:格式规范的情况下可准确识别章节，若论文中无章节，可能会识别有误。



毕业设计（论文）

题目 网络流量检测管理系统的设计与开发

学 院 计算机科学与工程学院

专 业 计算机科学与技术

班 级 122030701

学生姓名 郭子谊 学号 11912060207

指导教师 范伟 职称 教授

时 间 2026年06月

目 录

摘 要I

AbstractII

1 绪论1

1.1 研究背景与意义1

1.1.1 网络代理在开发调试与安全观测中的作用1

1.1.2 传统代理工具在可视化管理方面的不足1

1.1.3 本课题的工程实践意义	2
1.2 国内外研究现状	3
1.2.1 网络代理与抓包工具发展现状	3
1.2.2 Web 化运维控制台发展现状	4
1.2.3 对象存储在流量留存中的应用	4
1.3 研究目标与主要内容	5
1.3.1 系统建设目标	5
1.3.2 主要研究内容	5
1.3.3 论文拟解决的问题	6
1.4 论文组织结构	6
2 系统相关技术	7
2.1 HTTP/HTTPS 代理相关技术	7
2.1.1 HTTP 正向代理原理	7
2.1.2 HTTPS CONNECT 隧道原理	7
2.1.3 MITM 中间人代理原理	8
2.1.4 HTTP/1.1 长连接与分块传输	8
2.2 后端开发相关技术	9
2.2.1 Go 网络编程与 net/http	9
2.2.2 Go 并发控制技术	9
2.2.3 MinIO 对象存储技术	10
2.3 前端开发相关技术	10
2.3.1 Vue 3 与 Vite	10
2.3.2 Vue Router 与 Pinia	11
2.3.3 图表与虚拟滚动技术	11
2.4 前后端通信与部署相关技术	11
2.4.1 REST 配置接口	11
2.4.2 WebSocket 实时推送	12
2.4.3 前端嵌入式部署	12
2.5 设计模式相关技术	12
2.5.1 观察者模式（发布-订阅模式）	12
2.5.2 责任链模式	13
2.5.3 单例模式	13
2.5.4 适配器模式	14
3 系统需求分析	15

3.1	系统总体需求	15
3.1.1	用户角色与使用场景	15
3.1.2	功能性需求	15
3.1.3	非功能性需求	16
3.2	后端功能需求	17
3.2.1	代理转发需求	17
3.2.2	规则路由需求	17
3.2.3	访问控制需求	17
3.2.4	流量监控需求	18
3.2.5	抓包与对象存储需求	18
3.2.6	配置管理需求	18
3.3	前端功能需求	19
3.3.1	登录与代理核心连接	19
3.3.2	概览仪表盘	19
3.3.3	连接与历史连接管理	19
3.3.4	日志管理	19
3.3.5	MITM 抓包管理	20
3.3.6	路由、安全、存储和高级配置	20
3.4	数据与接口需求	20
3.4.1	配置数据模型	20
3.4.2	WebSocket 消息模型	21
3.4.3	存储数据模型	21
3.5	可行性分析	22
3.5.1	技术可行性	22
3.5.2	经济可行性	22
3.5.3	操作可行性	22
4	系统总体设计	23
4.1	系统总体架构设计	23
4.1.1	整体架构	23
4.1.2	前后端协作流程	23
4.1.3	运行时数据流	24
4.2	后端模块划分	24
4.2.1	代理核心模块	25
4.2.2	配置管理模块	25

4.2.3	路由模块	25
4.2.4	访问控制模块	25
4.2.5	流量监控与连接管理模块	26
4.2.6	MITM 抓包与存储模块	26
4.2.7	WebSocket 控制模块	26
4.3	前端模块划分	26
4.3.1	路由与布局模块	27
4.3.2	状态管理模块	27
4.3.3	页面展示模块	28
4.4	关键数据结构设计	28
4.4.1	代理上下文 Pcontext	29
4.4.2	连接信息 ConnectionInfo	29
4.5	系统安全设计	29
4.5.1	控制服务访问控制	29
4.5.2	MITM 使用边界	29
4.5.3	代理自环防护	30
5	系统详细设计与实现	30
5.1	后端启动流程实现	30
5.2	HTTP 代理实现	31
5.2.1	普通 HTTP 转发流程	31
5.2.2	HTTP-MITM 引擎实现	32
5.2.3	WebSocket 与流式响应处理	34
5.3	HTTPS 与 CONNECT 实现	35
5.3.1	CONNECT 隧道模式	35
5.3.2	MITM 模式	38
5.3.3	证书签发实现	39
5.4	路由引擎实现	40
5.4.1	出站拨号器设计	40
5.4.2	路由规则匹配	41
5.4.3	路由热重载	41
5.4.4	自环检测	42
5.5	访问控制实现	42
5.5.1	目标访问控制	42
5.5.2	来源 IP 封禁	42

5.5.3 拦截日志与连接关闭	43
5.6 流量统计与连接管理实现	43
5.6.1 HTTP 明文流量统计	43
5.6.2 隧道流量统计	45
5.6.3 连接生命周期管理	46
5.6.4 用户流量聚合	46
5.7 MITM 抓包与 MinIO 存储实现	47
5.7.1 抓包快照生成	47
5.7.2 请求体和响应体捕获	48
5.7.3 大文件与 chunked 上传处理	48
5.7.4 下载接口实现	49
5.8 WebSocket 控制服务实现	50
5.8.1 连接认证与订阅	50
5.8.2 实时数据推送器	51
5.9 前端控制台实现	51
5.9.1 登录页实现	51
5.9.2 仪表盘布局实现	52
5.9.3 概览页实现	52
5.9.4 连接页与历史连接页实现	53
5.9.5 日志页实现	53
5.9.6 MITM 页面实现	53
5.11 设计模式在系统中的应用	54
5.11.1 观察者模式（发布-订阅模式）的应用	54
5.11.2 责任链模式的应用	54
5.11.4 适配器模式的应用	54
6 系统测试与结果分析	55
6.1 测试环境与测试方法	56
6.1.1 测试环境	56
6.1.2 测试方法	56
6.2 单元测试设计	57
6.2.1 HTTP 解析器测试	57
6.2.3 访问控制测试	59
6.2.4 MinIO 存储测试	59
6.2.5 goroutine 泄露检查测试	59

6.3	功能测试设计	60
6.3.1	HTTP 代理功能测试	60
6.3.6	前端页面功能测试	60
6.4	性能测试设计	61
6.4.1	5MB 文件上传下载测试	61
6.4.2	高并发压力测试	61
6.4.3	pprof 性能分析	63
6.5	测试结果分析	66
6.5.1	功能完整性分析	66
6.5.2	性能与资源占用分析	66
6.5.3	问题与改进分析	66
7	总结与展望	66
7.1	工作总结	67
7.1.1	系统完成情况	67
7.2	系统特色	67
7.2.1	代理核心与控制台一体化	67
7.2.2	实时监控与可操作连接表	68
7.2.3	流式抓包与对象存储结合	68
7.3	不足之处	69
7.3.2	协议覆盖仍可扩展	69
7.3.3	数据持久化能力仍可增强	69
7.4	展望	70
7.4.1	多用户与权限管理	70
7.4.2	更丰富的规则体系	70
	致谢	70
	参考文献	71
	附录	72
	附录 A 配置文件示例	72
	附录 B 主要接口说明	75
	附录 C WebSocket 消息格式	77
	附录 D 测试与构建命令	79

摘 要

随着 Web 服务、接口联调和网络安全观测需求的增加，代理系统不仅需要完成请求转发，还需要具备实时监

控、规则配置、访问控制和抓包留存等管理能力。我们在该系统设计中后端主要采用 Go 语言实现，围绕 HTTP/HTTPS 代理转发流程完成核心功能开发。在基础代理能力方面，系统支持普通 HTTP 正向代理和 HTTPS CONNECT 隧道转发；在获得授权的测试环境中，还可以通过 MITM 方式对 HTTPS 流量进行明文解析，便于观察请求与响应内容。为了让代理过程具备可配置性和可追踪性，后端进一步实现了规则路由、访问控制、连接生命周期管理、实时流量统计以及抓包正文存储等功能。请求、响应和 CONNECT 处理过程被拆分为多条责任链，转发、路由判断、统计记录、正文捕获和安全检查等逻辑分别在对应节点中完成，从而降低了单个处理函数的复杂度。前端部分基于 Vue 3 构建，配合 Vite、Pinia、Vue Router、Chart.js 和虚拟滚动组件，实现了登录、概览、当前连接、历史连接、运行日志、MITM 抓包、路由配置、安全策略、用户监控、存储配置和高级设置等页面。系统的配置项主要通过 REST 接口完成读取与保存；运行过程中的流量、连接状态、日志、拦截记录、抓包记录以及用户流量统计，则通过 WebSocket 按主题推送到前端。对于请求体和响应体这类可能较大的数据，系统使用 MinIO 进行流式存储，避免代理进程和浏览器在大文件抓包场景下长时间占用过多内存。

测试工作主要围绕几个容易出问题的环节展开，包括 HTTP 解析器、配置加载与保存、访问控制规则、MinIO 存储流程、goroutine 泄露检查、HTTP/HTTPS 转发、MITM 抓包、KnownSize 与 Chunked 传输处理，以及一定规模下的压力基准测试。测试结果显示，系统能够完成代理转发、实时观测、策略配置和抓包留存等预期功能。虽然该系统仍更适合开发调试和毕业设计演示等中小规模场景，但从实现过程和测试结果来看，已经具备一定的可用性，也体现了 Go 后端并发处理、前后端实时通信和对象存储结合使用的工程实践价值。关键词：网络代理；Go；Vue；WebSocket；MITM；对象存储；访问控制

Abstract

As Web applications become more dependent on external services and complex network calls, a proxy tool used in development can no longer be limited to simple request forwarding. In practice, developers also need to see connection status, adjust routing rules, restrict risky access, and keep useful traffic records for later analysis. Based on this requirement, this thesis designs and implements a visual network proxy management system using Go and Vue. The system is mainly intended for development debugging, network traffic observation, and graduation project demonstration. Its workflow covers proxy access, runtime monitoring, policy adjustment, and data retention.

The backend is developed in Go and focuses on the processing path of HTTP and HTTPS traffic. It supports ordinary HTTP forward proxying and HTTPS CONNECT tunneling. In authorized test environments, it can also perform MITM inspection so that request and response contents can be observed in plaintext. Around these basic proxy functions, the backend further provides rule-based routing, access control, connection lifecycle management, real-time traffic statistics, and request and response body storage. To keep the forwarding logic maintainable, the system organizes request, response, and CONNECT handling through separate processing chains. Forwarding, route matching, traffic accounting, body capture, and security checks are therefore handled at different stages instead of being mixed in one large procedure.

The frontend is built with Vue 3, together with Vite, Pinia, Vue Router, Chart.js, and virtual scrolling. It provides pages for login, overview, active connections, historical connections, logs, MITM records, routing rules, security policies, user monitoring, storage settings, and advanced configuration. Configuration data is read and saved through REST APIs. Runtime data, including traffic statistics, connection updates, logs, interception records, MITM exchanges, and user traffic information, is pushed to the browser through WebSocket topics. For request and response bodies, especially when captured files are large, the system uses MinIO for streaming storage. This design avoids keeping large bodies in the proxy process or browser memory for a long time.

The evaluation covers several parts that are likely to affect system stability and correctness, including HTTP parsing, configuration management, access control, MinIO storage, goroutine leak detection, HTTP and HTTPS forwarding, MITM capture, known-size transmission, chunked transmission, and benchmark-based stress tests. The test results indicate that the system can complete the expected tasks of proxy forwarding, real-time observation, policy configuration, and capture retention. Although the current implementation is more suitable for development debugging and medium-scale demonstration scenarios than for large production deployments, it still shows practical engineering value in combining Go-based concurrent proxy processing, browser-side visualization, WebSocket communication, and object storage.

Key words: network proxy, Go, Vue, WebSocket, MITM, object storage, access control

1 绪论

1.1 研究背景与意义

1.1.1 网络代理在开发调试与安全观测中的作用

随着 Web 应用、移动应用和接口服务的规模不断扩大,客户端与服务端之间的通信链路已经从简单的页面访问扩展为接口调用、文件上传下载、长连接推送、第三方服务访问和多环境联调等多种形态。HTTP 协议以请求和响应为基本交互单元,不同 HTTP 版本共享统一的语义体系,请求方法、状态码和头部字段共同描述客户端意图与服务端结果[1]。在这一链路中,网络代理位于客户端与目标服务之间,既可以完成请求转发,也可以对连接状态、请求头、响应头、数据体大小和错误信息进行集中观测。

对于 HTTPS 场景,客户端通常使用 CONNECT 方法要求代理建立到目标主机的隧道,隧道建立成功后代理在两个方向上转发字节流[1]。这使代理工具能够在不破坏端到端加密的前提下完成访问转发;而在授权调试场景中,MITM 模式又可以通过本地证书将隧道内流量解析为明文 HTTP 请求和响应,从而辅助接口调试、故障定位和安全策略验证。WebSocket 则以单个 TCP 连接支持网页与服务端之间的双向通信,适合实时状态推送和在线协作等场景[2]。因此,一个完整的代理管理系统不仅要处理普通 HTTP 请求,还要处理 CONNECT 隧道、MITM 明文流和长连接流量。

本课题所实现的系统正是围绕上述需求展开。后端 CoreHttpServer 同时作为 HTTP 服务端和转发客户端工作,入口处根据请求方法区分普通 HTTP 请求和 CONNECT 请求;配置、路由、访问控制、流量统计、MinIO 存储

和 WebSocket 控制服务在主程序中按顺序初始化。该设计使系统既能承担网络代理的基础转发职责，也能为后续的连接列表、流量曲线、日志检索、抓包详情和用户流量统计提供统一数据来源。

1.1.2 传统代理工具在可视化管理方面的不足

HTTP 协议的交互过程通常可以拆分为请求和响应两个基本部分。虽然 HTTP/1.1、HTTP/2 等版本在连接复用、传输效率和报文组织方式上有所差异，但它们在请求方法、状态码和头部字段等语义层面保持了较强的一致性[1]。客户端通过方法和头部字段表达访问意图，服务端则借助状态码、响应头和数据体返回处理结果。在这条通信链路中，网络代理处在客户端和目标服务之间。它除了完成请求转发外，还能够把分散在不同连接中的状态信息集中起来，例如请求头、响应头、连接建立与关闭情况、数据体大小以及转发过程中的错误信息等。因此，代理系统不仅是一个中转组件，也可以作为开发调试和网络观测中的重要入口。

在 HTTPS 访问过程中，客户端一般会向代理发送 CONNECT 请求，要求代理与目标主机建立隧道。隧道建立成功后，代理主要负责在客户端和目标服务器之间转发字节流，本身并不直接理解加密后的业务内容[1]。这种方式能够在不破坏端到端加密关系的情况下完成访问转发，适合普通代理场景。对于已经获得授权的调试环境，代理还可以启用 MITM 模式，通过本地信任证书对隧道中的流量进行解密和重新加密，使 HTTPS 流量在代理侧还原为可分析的 HTTP 请求和响应。这样一来，开发人员就可以进一步检查接口参数、响应内容和异常返回，从而辅助接口调试、故障定位以及安全策略验证。另一方面，WebSocket 基于单个 TCP 连接提供网页端与服务端之间的双向通信能力，常用于实时状态推送、在线协作和持续连接类业务[2]。因此，面向实际使用的代理管理系统不能只处理普通 HTTP 请求，还需要同时覆盖 CONNECT 隧道、授权 MITM 明文流以及长连接流量。

本课题实现的系统也是围绕上述需求进行设计的。后端中的 CoreHttpServer 同时承担 HTTP 服务端和转发客户端的角色，请求进入系统后，入口逻辑会先根据请求方法区分普通 HTTP 请求和 CONNECT 请求，再交由对应的处理流程完成转发和记录。系统启动时，配置模块、路由模块、访问控制模块、流量统计模块、MinIO 存储模块以及 WebSocket 控制服务会按依赖关系依次初始化。通过这种组织方式，系统一方面能够完成网络代理最基本的转发任务，另一方面也能持续沉淀运行数据，为前端的连接列表、流量曲线、日志检索、抓包详情和用户流量统计等功能提供统一的数据来源。

1.1.3 本课题的工程实践意义

本课题具有较强的工程综合性。后端部分基于 Go 语言实现，使用 net/http 提供的服务端、客户端和传输层能力完成代理监听与请求转发，Go 标准库文档也明确提供了 HTTP 客户端和服务端实现[3]。项目代码中，main.go 创建代理核心并启动 HTTP 代理端口、HTTPS 代理端口和控制服务端口；CoreHttpServer 维护请求处理器、响应处理器、CONNECT 处理器、路由器、访问控制器、MinIO 客户端和连接表。围绕这些结构，系统需要处理 TLS、连接劫持、HTTP 解析、并发统计、通道推送和流式数据复制等问题。

前端部分基于 Vue 3、Vite、Pinia、Vue Router、Chart.js 和虚拟滚动技术实现。Vue 官方文档将 Vue 定义为构建用户界面的 JavaScript 框架，并强调其声明式、组件化和响应式模型[4]。本项目的前端路由将登录页和控制台页面分离，控制台页面包含概览、连接、日志、MITM、路由配置、存储配置和安全策略等子页面；Pinia Store 负责维护 WebSocket 连接状态、实时流量、连接快照、日志、抓包记录、拦截日志和用户流量列表。前后端结合后，可以较完整地体现现代 Web 管理台的组件化开发方式。

从软件工程训练角度看，本课题覆盖了需求分析、架构设计、模块划分、接口设计、异常处理、测试验证和部署构建等多个环节。系统既有明确的网络协议基础，也有可运行的前后端实现；既关注普通请求转发，也关注流量观测、访问控制和大对象存储等工程问题。因此，该课题不仅能够展示开发语言和框架的使用能力，还能体现对系统边界、运行状态和可靠性问题的综合分析能力。

1.2 国内外研究现状

1.2.1 网络代理与抓包工具发展现状

目前一些成熟代理工具已经提供了较完整的功能，例如请求查看、规则匹配、证书安装、响应改写、限速控制、脚本扩展和日志分析等。这类工具在实际开发和测试中很有价值，但对毕业设计而言也存在两个问题：一方面，功能体系较大，初学者往往需要较长时间才能理解其内部工作过程；另一方面，很多工具更强调使用效果，底层实现链路并不一定适合在课程项目中逐步展示。我们在该系统设计中没有把目标设定为复刻商业化代理工具，而是将重点放在一条可解释、可演示、可验证的核心链路上。普通 HTTP 请求进入系统后由责任链逐步处理，CONNECT 请求则通过策略选择器判断是直接透传还是进入 MITM 流程；访问控制和路由规则支持通过配置热重载生效，运行中的连接、日志、流量和抓包数据则通过 WebSocket 统一推送到前端页面。

基于上述考虑，本课题在相关研究和已有工具中的定位更偏向工程化实现与可视化整合。系统并不试图覆盖所有网络协议，也不追求实现所有高级抓包能力，而是围绕代理核心、规则系统、实时监控、抓包留存和前端控制台之间的配合关系展开设计。这样的定位更适合毕业设计的论证方式：后续章节可以把需求分析、模块设计、代码实现和运行结果对应起来，说明每一项功能是如何落到具体模块和处理流程中的，而不是只停留在对现成工具功能的介绍上。

1.2.2 Web 化运维控制台发展现状

随着系统运行数据规模增加，运维和调试工具逐渐从命令行输出转向 Web 化控制台。Web 控制台的作用并不只是把后端接口简单展示出来。一个可用的控制台通常需要包含实时仪表盘、配置管理、日志查询、连接列表和告警信息等页面，帮助使用者在较短时间内判断系统当前是否正常运行。对于代理系统来说，连接建立、连接关闭、流量变化和错误日志都具有较强的实时性。如果前端只通过定时调用 REST 接口获取数据，就可能出现状态更新滞后、接口请求过于频繁以及页面显示与后端实际状态不一致等问题。

WebSocket 协议基于单个 TCP 连接提供双向通信能力，相比轮询方式更适合网页端与服务端之间的实时消息交换[2]。在我们在该系统设计中，后端控制服务会在 /start 路径将普通 HTTP 连接升级为 WebSocket 连接。前端连接建立后，会主动发送订阅消息，说明当前页面或模块需要接收的数据主题，包括 traffic、connections、logs、mitm_detail、intercept_logs 和 user_traffic 等。通过这种方式，前端不需要反复请求全部数据，而是根据订阅主题接收后端推送的运行状态更新。后端 Hub 根据订阅主题批量推送数据，前端 Store 接收后再通知对应页面刷新。

项目的前端页面也体现了 Web 化控制台的发展方向。概览页展示最近 60 秒上传和下载速率曲线、安全策略摘要、路由策略摘要和用户流量摘要；连接页和历史连接页支持筛选、排序、展开和关闭连接；日志页支持等级和类别过滤；MITM 页面支持搜索、排序、展开 Header 与下载 Body。与单页数据展示相比，这种多页面控制台更符合代理管理系统的日常使用方式。

1.2.3 对象存储在流量留存中的应用

在抓包和流量审计场景中，请求体与响应体可能远大于头部和元数据。如果系统将所有 Body 直接保存在内存中，不仅会增加代理进程的内存压力，也会导致前端 WebSocket 消息过大，影响实时界面稳定性。更合理的方式是将头部、状态码、耗时、大小和对象 Key 作为元数据推送到前端，而将真正的大对象写入独立存储系统。

MinIO Go Client SDK 提供面向 Amazon S3 兼容对象存储的访问接口，并包含对象上传、下载、状态查询和预签名 URL 等对象操作能力[5]。我们在该系统设计中后端使用 MinIO 保存 MITM 模式下捕获的请求体和响应体。已知 Content-Length 的数据可以通过流式上传写入对象存储，未知长度或 chunked 数据先写入临时文件，在获得实际大小后再上传。这样既能保留抓包正文，又避免将大文件完整加载到内存中。

这种设计与本课题的可视化目标相匹配。前端 MITM 页面主要展示会话 ID、父会话 ID、方法、Host、URL、状态码、耗时、请求大小、响应大小和上传状态；当用户确实需要正文内容时，再通过 /api/storage/download 根据对象 Key 获取下载链接或经控制服务代理下载。由此，系统在实时性、内存占用和数据留存之间取得了较好的平衡。

1.3 研究目标与主要内容

1.3.1 系统建设目标

我们设置该系统的总体目标是设计并实现一套基于 Go 与 Vue 的可视化网络代理管理系统。系统不仅要完成网络代理最基本的请求转发任务，还需要通过 Web 控制台把运行状态、连接变化和配置项展示出来，使使用者能够在浏览器中完成观察和管理。后端部分主要面向普通 HTTP 代理、HTTPS CONNECT 隧道以及授权场景下的 MITM 解析；前端部分则围绕代理核心连接、实时概览、连接列表、日志查看、MITM 抓包、安全策略、路由配置、存储配置和高级参数配置等功能展开。

1.3.2 主要研究内容

我们的研究内容首先包括网络代理核心设计。代理核心是我们在该系统设计中后端实现中最基础的一部分，其主要逻辑围绕 CoreHttpServer 展开。我们必须避免把不同类型的处理逻辑集中在同一个函数中，系统将普通请求、响应处理和 CONNECT 请求分别组织为请求责任链、响应责任链和 CONNECT 处理链。普通 HTTP 请求进入后由 MyHttpHandle 完成转发处理；HTTPS 场景下的 CONNECT 请求则交给 MyHttpsHandle 判断和处理。当系统启用授权 MITM 模式时，后端会结合协议嗅探、TLS 握手、HTTP/1.1 请求读取和响应写回等步骤，将隧道中的流量还原为可分析的明文 HTTP 交互。

1.3.3 论文拟解决的问题

本文拟解决的第一个问题是如何在同一后端程序中统一处理普通 HTTP 请求、CONNECT 隧道和 MITM 明文请求。三类流量在协议形态、连接生命周期和可观测粒度上存在差异：普通 HTTP 请求可以直接读取请求和响应，CONNECT 隧道只能统计加密字节流，MITM 模式则需要在授权前提下重新建立 TLS 会话并解析 HTTP 消息。论文后续将说明系统如何通过处理链和上下文对象统一组织这些流程。

第二个问题是如何在代理转发过程中完成准确且低侵入的流量统计。代理不能为了统计而提前读取完整 Body，否则会影响流式转发和大文件场景。因此，系统采用 Reader 包装方式，在请求体和响应体被正常读取时同步累计字节数，并将统计结果写入全局流量、连接表 and 用户 Host 聚合结构。该方式能够保持主链路转发语义，同时提供前端实时展示所需数据。

第三个问题是如何在不阻塞主转发链路的前提下留存抓包正文。系统需要把请求体和响应体保存到 MinIO，但上传失败不应导致代理整体不可用。为此，系统将抓包元数据与 Body 对象分离，使用 Pipe 和上传协程处理已知长度数据，使用临时文件处理未知长度数据，并在存储不可用时保留请求响应元数据。最后，论文还将讨论如何将高频运行数据稳定推送到前端，以及如何设计可热更新的路由与访问控制规则。

1.4 论文组织结构

本文共分为七个主要章节，并在正文之后保留致谢、参考文献和附录。第 1 章为绪论，说明课题背景、研究意义、国内外相关现状、建设目标、主要研究内容和论文结构。第 2 章介绍系统相关技术 第 3 章进行系统需求分析，从用户角色、功能性需求、非功能性需求、后端功能、前端功能、数据与接口需求以及可行性等方面明确系统边界。第 4 章给出系统总体设计，第 6 章为系统测试与结果分析，第 7 章总结系统完成情况和论文工作

2 系统相关技术

2.1 HTTP/HTTPS 代理相关技术

2.1.1 HTTP 正向代理原理

HTTP 正向代理工作在客户端和目标服务器之间。客户端访问外部资源时，会把包含完整目标地址的 HTTP 请求发送给代理服务器，再由代理服务器代替客户端向目标服务器发起请求，并把服务端响应返回给客户端。HTTP 语义规范将请求方法、目标资源、头部字段、状态码和响应内容组织在统一的请求响应模型中，这使代理服务器能够在遵循协议语义的前提下解析、修改和转发消息[1]。在正向代理场景中，代理并不只是完成一次网络中转，它还可以在请求进入和响应返回两个阶段记录连接状态、头部大小、数据体大小以及转发过程中的异常信息。

HTTP 正向代理的实现需要同时考虑透明性和可观测性。透明性要求代理尽量不改变原始请求的业务语义，例如不能把服务端内部使用的 RequestURI 字段直接带入客户端请求；可观测性则要求系统能够记录请求体和响应体大小，但又不能为了统计而提前把完整数据读入内存。我们在该系统设计中采用 Reader 包装方式处理这一问题：当请求体或响应体在正常转发过程中被读取时，包装器同步累加已读取的字节数。这样，流量统计与实际转发过程保持一致，也不会破坏上传、下载等流式传输场景中的 HTTP 请求响应链路。

2.1.2 HTTPS CONNECT 隧道原理

HTTPS 正向代理通常使用 CONNECT 方法建立隧道。在 HTTPS 代理访问中，客户端通常会先向代理服务器发送类似 CONNECT host:port HTTP/1.1 的请求。代理收到该请求后，并不会直接解析 HTTPS 内部的业务报文，而是先与目标服务器建立 TCP 连接；连接建立成功后，再向客户端返回隧道建立成功的响应。此后，客户端与目标服务器之间的 TLS 握手数据以及后续应用层数据，都会通过这条隧道进行传输[1]。在普通隧道透传模式下，代理能够接触到的只是加密后的字节流，因此无法直接获得 HTTP 方法、访问路径、请求头部或响应状态等明文信息。

我们在该系统设计中，CONNECT 隧道请求由 MyHttpsHandle 负责处理。该函数在进入具体策略前，会先通过 http.Hijacker 从 Go 的 HTTP 服务端接管底层网络连接，使后续逻辑可以直接操作客户端连接。随后，系统会结合当前配置和 CONNECT 处理链判断本次请求应采用透传、MITM 还是拒绝策略。若最终选择透传模式，后端会调用路由拨号器连接目标主机，并向客户端写入 HTTP/1.0 200 Connection established，表示隧道已经建立。之后，系统通过双向复制在客户端连接和目标连接之间转发数据，使 TLS 握手和加密应用数据能够继续在隧道中完成传输。为了便于前端展示，系统还把隧道登记为连接表中的 TUNNEL 或 HTTPS-Tunnel 记录，并用包装后的连接统

计上行和下行字节数。

隧道代理的关键特点是连接生命周期长、数据边界不再等同于单个 HTTP 请求。系统在该模式下不生成明文抓包记录，而是把流量统计粒度限定在隧道整体层面。这样既符合 HTTPS 端到端加密的基本语义，也为用户在控制台观察连接建立、流量变化和连接关闭提供了必要信息。

2.1.3 MITM 中间人代理原理

MITM 模式是在授权调试环境下对 HTTPS 流量进行明文观测的一种代理方式。其基本过程是：客户端信任本地代理生成的 CA 证书，代理在客户端侧为目标域名动态签发证书并终止 TLS 连接，同时在服务端侧继续与真实目标服务器建立连接。这样，代理可以在中间位置读取明文 HTTP 请求和响应。MITM 模式能够让代理侧看到 HTTPS 流量中的明文请求和响应，但它同时也改变了客户端到服务端之间原本的加密边界。因此，该功能不能作为普通访问场景下的默认能力使用，只应限定在用户已经明确授权的开发调试、课程演示或内部测试环境中，用于接口分析、故障定位和安全策略验证。

请求体和响应体在满足条件时会被进一步包装为 MinIO 捕获读取器，前端 MITM 页面只接收方法、URL、Host、头部、状态码、耗时、大小和对象 Key 等元数据。这样既保留了明文分析能力，又避免将大体积正文直接推送到浏览器。

2.1.4 HTTP/1.1 长连接与分块传输

HTTP/1.1 允许在同一条 TCP 连接上复用多个请求，这给代理服务器的实现带来了一定复杂度。对于普通服务端程序来说，标准库通常已经完成了第一个请求的解析；但代理在 Hijack 底层连接之后，如果连接中还会继续出现后续请求，就不能只处理当前这一次请求，而需要继续从连接流中读取和分发。与此同时，分块传输、SSE 和 WebSocket 等机制也会影响响应处理方式。分块响应通常没有固定的 Content-Length，SSE 对刷新时机比较敏感，WebSocket 在协议升级成功后则不再按照普通 HTTP 响应体进行处理。

针对这些情况，我们在该系统中在不同代理流程中分别加入了对应处理逻辑。在 HTTP-MITM 和 HTTPS-MITM 流程中，`http1parser.RequestReader` 负责从连接流中循环读取 HTTP 请求，使同一连接上的多个请求能够被连续处理。

2.2 后端开发相关技术

2.2.1 Go 网络编程与 net/http

Go 标准库中的 `net/http` 同时覆盖了 HTTP 服务端、客户端以及底层传输相关能力，因此比较适合用来实现代理这类中间层程序[3]。代理程序一方面需要接收来自客户端的请求，另一方面又要根据请求内容继续访问目标服务器，这与 `net/http` 提供的 `Handler`、`Client` 和 `Transport` 等能力正好对应。我们在该系统中后端以 `CoreHttpServer` 作为代理核心结构体。它对外实现 `http.Handler` 接口，用于接入 Go 的 HTTP 服务端；对内则维护 `http.Transport`、请求处理器、响应处理器、CONNECT 处理器、路由器、访问控制器、MinIO 客户端以及连接表等对象。通过这种组织方式，`CoreHttpServer` 既承担入口服务端的职责，也负责发起出站请求。

2.2.2 Go 并发控制技术

代理系统天然具有高并发特征：多个客户端连接会同时进入代理核心，连接表、流量计数、日志通道、抓包记录和 WebSocket 推送都需要在并发环境下工作。Go 的 `sync` 包提供互斥锁、读写锁、条件变量和并发 Map 等基础

同步原语，其中 `sync.Map` 适合多协程读写的连接表或订阅表场景[6]。我们在该系统设计中在后端连接管理和 WebSocket Hub 中大量使用 `sync.Map`，在访问控制、配置管理、路由器和用户流量聚合中使用 `sync.RWMutex` 控制读写临界区。

系统的计数类数据主要使用原子操作。全局上传下载流量由 `atomic.Int64` 保存，连接会话号和 MITM 交换记录编号通过 `atomic.AddInt64` 递增，累计拦截次数使用 `atomic.Uint64`。这类统计变量通常更新频率较高，而且每次更新只涉及较小的数值变动。如果在每次累加时都使用互斥锁，容易在高并发转发场景下增加锁竞争。因此，系统在这部分采用原子操作完成计数更新。对于用户 IP 到 Host 维度的流量统计，后端会先取得对应的统计对象，再分别对上行和下行字段执行原子累加。这样可以使流量记录尽量贴近实际转发过程，同时避免频繁统计逻辑长时间占用主转发链路。

2.2.3 MinIO 对象存储技术

我们在完成抓包逻辑时需要处理请求体和响应体，其中正文可能是图片、压缩包、视频片段或大文件。如果所有正文都直接保存在内存中，代理进程和浏览器端都会面临较大压力。MinIO 提供兼容 Amazon S3 的对象存储能力，并提供 Go 客户端 SDK 进行对象上传、下载和状态管理[5]。我们在该系统设计中将 MITM 抓包正文保存到 MinIO，对前端只传递对象 Key、大小、上传状态和错误信息。

在实现上，`myminio.Client.BuildBodyReader` 会把原始 Body 包装为 `bodyCaptReader`。主转发链路读取 Body 时，数据一方面继续流向目标服务器或客户端，另一方面写入 `io.Pipe`，上传协程从 Pipe 中读取并写入对象存储。对于已知 `Content-Length` 的数据，系统可以直接按固定大小调用上传接口；对于未知长度或 `chunked` 数据，系统先写入临时文件，获得实际大小后再上传，并在结束后清理临时文件。

下载能力通过控制服务中的 `/api/storage/download` 暴露。若配置了 `PublicEndpoint`，系统可以生成面向对象存储的访问地址；若没有公开访问端点，则通过控制服务代理下载对象。该方式把抓包元数据展示和大对象获取分离，既保证 MITM 页面加载轻量，也允许用户需要时取回完整请求体或响应体。

2.3 前端开发相关技术

2.3.1 Vue 3 与 Vite

Vue 3 是以声明式渲染和组件化为核心的前端框架，适合将复杂管理平台拆分为多个可维护的页面组件[4]。我们在该系统设计中前端使用 Vue 3 组织登录页、仪表盘布局、概览、连接、历史连接、日志、MITM、路由配置、存储配置、高级配置、安全策略和用户监控等页面。

2.3.2 Vue Router 与 Pinia

Vue Router 是 Vue 官方客户端路由方案，可在单页应用中将浏览器地址与页面组件对应起来，并通过 `RouterView` 渲染当前路由组件[8]。本项目前端路由在 `src/router/index.js` 中定义。系统通过 `createRouter` 和 `createWebHistory` 创建路由实例，其中根路径对应登录页，`/dashboard` 对应控制台整体布局。在控制台布局之下，前端进一步划分出概览、连接、历史连接、日志、MITM、路由配置、高级配置、存储配置和安全策略等子路由，使不同功能页面能够在统一框架内切换。

2.3.3 图表与虚拟滚动技术

代理管理平台需要展示实时上传速率、下载速率和累计流量。`Chart.js` 提供基于 Canvas 的图表绘制能力，支持

在页面中创建折线图、柱状图等常见可视化图形[10]。我们在该系统设计中在概览页中注册 `Chart.js` 组件，并通过 `new Chart` 创建最近 60 秒上传和下载速率折线图。`WebSocket Store` 每收到一次 `traffic` 消息，就把数据写入 `trafficHistory` 并通知概览页更新图表。

连接列表、日志列表和 MITM 抓包列表都可能随着系统运行不断增长。如果直接渲染所有 DOM 节点，浏览器会在排序、筛选和滚动时承受较大压力。`TanStack Virtual` 的 `Vue` 适配器提供 `useVirtualizer`，用于根据滚动容器和条目估算尺寸只渲染可视区域附近的元素[11]。本项目在概览连接简表、详细连接页、历史连接页、日志页、MITM 页面和用户监控页中使用虚拟滚动，以提高大列表展示的流畅度。

图表和虚拟滚动共同服务于实时控制台体验。图表让用户快速判断代理当前负载和流量趋势，虚拟滚动让用户在高频连接和大量日志下仍能进行筛选、排序和展开操作。二者都不改变后端代理逻辑，但直接影响系统可观测性的使用效果，是前端技术选型中的重要部分。

2.4 前后端通信与部署相关技术

2.4.1 REST 配置接口

REST 接口更适合承载配置读取、配置保存和文件下载这类边界明确、调用频率相对较低的操作。与实时推送数据不同，这些请求通常有清晰的输入和输出结果，也不需要长期保持消息通道。我们在该系统设计中的控制服务在 `StartControlServer` 中注册了 `/api/config`、`/api/stats` 和 `/api/storage/download` 等接口，用于处理前端控制台中的配置管理、统计补偿和抓包对象下载需求。

2.4.2 WebSocket 实时推送

`WebSocket` 协议在单个 `TCP` 连接上提供双向通信能力，适合浏览器与服务端之间的实时消息交换[2]。代理管理系统中的流量速率、连接表、日志、拦截记录、MITM 交换详情和用户流量都具有实时性，如果完全依赖轮询接口，会增加请求数量并降低状态同步效率。基于运行数据实时性较强的特点，我们在该系统设计中将 `WebSocket` 作为后端向前端推送状态数据的主要通道。相比不断轮询 REST 接口，`WebSocket` 能够在同一条连接上持续传输消息，更适合连接变化、流量统计、日志批次和抓包记录这类需要及时更新的数据。

2.4.3 前端嵌入式部署

为了方便毕业设计演示和单程序部署，我们在该系统设计中将前端构建产物嵌入 Go 后端。`Go` 的 `embed` 包支持在编译期把文件或目录嵌入程序，嵌入后的 `embed.FS` 可与 `net/http` 等包协同提供静态文件服务[12]。本项目在 `proxysocket/proxy.go` 中使用 `//go:embed dist` 嵌入前端 `dist` 目录，并通过 `http.FileServer` 对外提供静态资源。

控制服务同时承担静态页面、REST 接口和 `WebSocket` 服务三类职责。在控制服务处理前端页面访问时，`handleStaticFiles` 会先尝试查找并返回真实存在的静态资源。如果访问路径没有对应文件，并且该路径也不是带有扩展名的静态资源请求，后端就返回 `index.html`。通过我们这样处理后，前端路由可以继续由 `Vue Router` 的 `History` 模式接管。用户直接访问或刷新 `/dashboard/overview` 等控制台路径时，浏览器仍能回到单页应用入口，再由前端根据当前路径渲染对应页面。

2.5 设计模式相关技术

2.5.1 观察者模式（发布-订阅模式）

观察者模式用于建立一个对象状态变化与多个依赖对象之间的一对多通知关系，发布-订阅可以看作观察者思想在消息主题上的工程化表达[13]。我们在该系统设计中的实时推送正是这一思想的应用：后端代理核心、日志模块、抓包模块和用户流量模块不断产生运行数据，WebSocket Hub 根据客户端订阅主题把数据分发给不同前端页面。

在后端，WebSocketHub 保存连接与订阅信息，broadcastToTopic 根据主题判断是否应向某个客户端发送消息。StartTrafficPusher、StartConnectionPusher、StartLogPusher、StartMitmDetailPusher、StartInterceptLogPusher 和 StartUserTrafficPusher 分别对应不同数据源。在前端，Pinia Store 接收消息后再调用页面注册的回调，例如概览页订阅流量和连接，日志页订阅日志，MITM 页面订阅抓包记录。通过这种两层观察者结构，代理核心不需要直接依赖具体页面组件。

2.5.2 责任链模式

责任链模式将多个处理步骤按顺序串联，请求在链上传递，每个处理器可以选择继续传递或提前返回结果[13]。代理服务器非常适合使用这一模式，因为一次请求可能依次经过访问控制、流量统计、MinIO 捕获、路由选择、抓包快照、响应包装和日志记录等环节。如果把这些逻辑集中写在一个函数中，会导致入口函数过长且难以维护。

我们在该系统设计中在 CoreHttpServer 中维护请求处理器、响应处理器和 CONNECT 处理器三类链。filterRequest 顺序执行请求处理器，当访问控制命中规则时可以直接返回 403 响应；filterResponse 顺序执行响应处理器，用于统计响应头、包装响应体并在关闭时发送 MITM 交换记录；CONNECT 处理器则决定隧道是透传、MITM 还是拒绝。路由、访问控制和流量监控均通过 HookOnReq、HookOnResp 或 DoConnectFunc 注入，不需要修改代理入口的核心分发结构。

2.5.3 单例模式

单例模式强调某类全局状态或服务在系统运行期只有一个统一访问入口[13]。Go 语言项目中不一定通过传统私有构造函数实现单例，更常见的方式是使用包级变量、初始化函数和显式注入控制全局唯一实例。我们在该系统设计中，全局流量计数器、日志通道、拦截日志通道、MITM 交换通道、用户流量监控器和 WebSocket Hub 都体现了这一思想。

例如，GlobalTrafficUp 和 GlobalTrafficDown 保存进程级累计流量，所有请求和隧道统计都会写入这两个计数器；GlobalExchangeChan 是 MITM 抓包记录进入控制服务的统一通道；GlobalUserTraffic 维护客户端 IP 到 Host 维度的流量聚合；hub 作为控制服务中的唯一 WebSocket Hub，统一管理前端连接和主题订阅。通过集中入口，前端展示的数据来源保持一致，也便于后续测试和问题定位。

单例式全局状态需要注意并发安全。我们在该系统设计中对高频计数使用原子变量，对复杂集合使用锁或 sync.Map，对通道写入采用容量限制和非阻塞分支。这样可以避免全局状态成为并发瓶颈，同时保持运行数据在各模块之间的可访问性。

2.5.4 适配器模式

适配器模式用于把不同接口或不同实现包装为统一接口，使上层逻辑可以用一致方式调用[13]。我们在该系统设计中的规则路由模块使用 OutboundDialer 抽象出站方式，DirectDialer 和 HttpProxyDialer 分别代表直连和

二级 HTTP 代理，但都实现 Dial、Name 和 GetTransport。路由器只需要根据规则返回某个 OutboundDialer，不必关心具体出站方式。

RouterRoundTripper 也是适配器模式的体现。MITM 明文请求最终仍通过 RoundTrip 发起出站访问，而路由选择本身发生在拨号器层。RouterRoundTripper 把路由匹配和对应节点的 Transport 调用适配为统一的 RoundTrip 接口，使 MITM 流量也能复用规则路由能力。这样普通请求、MITM 请求和隧道透传可以在不同层次上共享路由逻辑。

流量统计和 MinIO 捕获中的 Reader 包装同样体现了适配思想。原始 io.ReadCloser 只负责读取数据，系统通过 reqBodyReader、respBodyReader 和 bodyCaptReader 为其增加流量累计、父子连接统计、用户流量聚合和对对象存储上传能力，但对外仍表现为可读取、可关闭的 Body。上层转发逻辑继续使用 io.Copy 或 Transport.RoundTrip，无需感知底层已经叠加了统计和存储功能。

3 系统需求分析

3.1 系统总体需求

3.1.1 用户角色与使用场景

我们在该系统设计中面向的主要使用者包括开发调试人员、课程演示人员以及需要观察网络访问状态的网络管理员。不同人使用时对代理系统的关注点并不完全相同。由于代理系统处在客户端与目标服务之间，其使用边界也必须在需求阶段明确。普通 CONNECT 隧道只承担加密字节流转发，符合 HTTPS 端到端加密的基本语义；MITM 模式会终止客户端侧 TLS 连接并读取明文 HTTP 内容，因此只能用于用户明确授权的开发、测试和课程演示环境。系统在功能上提供 MITM 能力，但论文需求分析中将其定位为受控调试能力，而不是默认启用的通用流量解析能力。

3.1.2 功能性需求

通过我们自己仔细思考评估，系统的功能性需求主要围绕代理转发、实时观测、规则控制、抓包留存、配置管理和前端交互几个方面展开。

表 3-1 系统功能性需求表

编号	需求项	需求说明
F-01	代理转发	支持普通 HTTP 请求转发、HTTPS CONNECT 隧道透传、HTTP-MITM 和 HTTPS-MITM 明文处理。
F-02	实时监控	统计全局流量、活动连接、历史连接、运行日志、拦截日志和用户 IP 到 Host 维度流量。
F-03	规则路由	支持二级 HTTP 代理节点，按域名后缀、域名关键词和 IP 规则选择出站路径。
F-04	访问控制	支持目标域名或 IP 拦截、来源 IP 封禁、HTTP 与 CONNECT 入口统一拦截。

F-05	MITM 抓包	在授权场景下记录请求、响应、状态码、耗时、头部、大小和正文对象 Key。
F-06	配置管理	支持端口、MITM、路由、访问控制、MinIO、PublicIPs 和高级行为开关配置。
F-07	连接操作	支持关闭全部连接、关闭单条连接、按来源 IP 断开连接和清理离线用户流量记录。

3.1.3 非功能性需求

性能需求主要体现在大文件上传下载、高频日志推送和多连接并发这几类场景中。代理服务器不能为了统计流量或保存抓包内容，就提前把完整请求体、响应体全部读入内存，否则既会破坏流式转发语义，也会明显增加内存压力。因此，系统要求流量统计和对象存储捕获都通过 Reader 包装方式完成，在主转发链路正常读取数据时同步累计字节数或写入上传管道。WebSocket 推送同样需要进行批量化处理，避免日志和抓包记录逐条推送，导致浏览器端频繁渲染和状态更新。

稳定性需求要求系统在连接关闭、客户端中断、目标服务异常、MinIO 不可用以及配置变更等情况下，仍然尽量保持代理核心可用。代码中，ConnectionInfo 保存了 OnClose 回调，便于用户从前端主动关闭异常连接；当 MinIO 初始化失败时，程序不会直接退出，而是将存储能力降级为不可用状态，保证代理转发仍可继续运行。日志、拦截日志和抓包交换记录则采用带容量的通道，并配合非阻塞发送策略，降低辅助功能反向阻塞代理主链路的风险。

可维护性需求要求系统各模块边界相对清晰。代理核心主要负责请求分发和处理链执行，路由、访问控制、流量统计、MinIO 捕获和 WebSocket 推送则通过独立模块接入。前端也按照业务功能拆分为概览、连接、日志、MITM、路由、安全、存储和高级配置等视图，避免所有交互逻辑集中在单一页面中。可扩展性方面，系统后续应能够继续增加规则类型、推送主题、页面视图和存储策略，而不需要重写代理入口。安全性方面，控制服务认证、MITM 开关、访问控制规则和公网 IP 防自环配置都需要有明确边界。目前控制服务采用简单 token 认证，能够满足课程演示和本地调试场景，但后续仍有必要扩展账号体系、权限控制和操作审计能力。

3.2 后端功能需求

3.2.1 代理转发需求

后端代理转发需求是我们在该系统设计中的基础。系统会自动识别进入代理端口的请求类型：当请求方法为 CONNECT 时进入隧道处理流程，当请求不是 CONNECT 时进入普通 HTTP 处理流程。我们查阅RFC文档CONNECT 是 HTTP 语义中专门用于建立隧道的机制[1]，代理在透传模式下不能解析加密后的 HTTP 内容，因此系统只统计隧道整体上行和下行字节数，并把连接以 TUNNEL 或 HTTPS-Tunnel 形式登记到连接表中。

MITM 需求要求系统在授权场景下解析隧道内明文请求。系统需要根据配置选择 CONNECT 透传、HTTPS-MITM 或 HTTP-MITM 策略；在 MITM 模式下为目标主机生成证书，完成客户端侧 TLS 握手，再从解密后的连接中循环读

取 HTTP/1.1 请求。由于同一条连接可能承载多个请求，需求中还要求支持长连接、分块响应、SSE、WebSocket 升级和 HEAD 响应等特殊场景，避免代理在流式响应或协议升级时错误捕获 Body。

3.2.2 规则路由需求

后端代理转发是我们在该系统设计中所有功能的基础。系统首先需要识别进入代理端口的请求类型，根据不同类型进入到不同的处理器分支中，通过路由规则匹配到不同的路由器。每一个路由器会绑定一个固定的下一跳路由。

3.2.3 访问控制需求

访问控制需求可以分为目标访问控制和来源访问控制两类。目标访问控制主要面向请求目的地，系统需要支持域名后缀匹配、域名关键词匹配以及 IP 精确匹配，用于限制某些目标站点或地址的访问。来源访问控制则面向客户端侧，主要通过来源 IP 封禁来限制异常或不允许访问的客户端。为了便于在安全策略页面进行管理和演示，两类规则都应提供独立的启停开关，并允许用户填写备注字段，说明规则用途或测试场景。

3.2.4 流量监控需求

流量监控需要覆盖全局、连接和用户三个层级。第一层是全局流量，包括进程级累计上传、累计下载，以及按秒计算的上传和下载速率，用于支撑概览页面的总体趋势展示。第二层是连接流量，记录单个连接的请求头、请求体、响应头、响应体和总流量，主要服务于活动连接和历史连接页面。

3.2.5 抓包与对象存储需求

抓包需求只在 MITM 开启且用户授权的场景下生效。系统应为每个被解析出的明文请求生成一次 HttpExchange 记录，包含交换编号、子会话编号、父隧道编号、时间、请求快照、响应快照、耗时和错误信息。请求快照需要保存方法、URL、Host、头部和请求大小，响应快照需要保存状态码、状态文本、头部和响应大小。对于被本地访问控制拦截、WebSocket 升级或需要跳过的流式响应，系统不应错误生成完整抓包记录。

对象存储需求来源于大 Body 场景。请求体和响应体可能远大于头部元数据，不能直接通过 WebSocket 推送到浏览器。系统需要将正文保存到 MinIO，并在抓包记录中只保留对象 Key、大小、上传状态、Content-Type 和错误信息。MinIO Go Client SDK 提供了面向 S3 兼容对象存储的上传和下载能力[5]，我们在该系统设计中基于该能力实现流式上传。对于已知 Content-Length 的数据，系统按已知长度上传；对于未知长度或 chunked 数据，系统先写入临时文件，获得实际大小后再上传，上传完成后清理临时文件。

3.2.6 配置管理需求

配置管理需求主要解决系统参数如何保存、读取和运行时生效的问题。系统需要持久化保存的内容包括代理端口、HTTPS 端口、MITM 开关、HTTP-MITM 行为、路由规则、代理节点、访问控制规则、来源 IP 封禁、PublicIPs、MinIO 连接信息以及高级代理行为开关等。配置文件采用 JSON 格式保存，程序启动时会先读取已有配置，并对缺失字段补全默认值；如果配置文件不存在，系统应自动写入一份默认配置，减少首次运行时的手动准备工作。

3.3 前端功能需求

3.3.1 登录与代理核心连接

前端登录页需要提供控制服务地址、密钥和备注输入能力，并支持保存多个代理核心连接信息。用户连接成功

后，前端将 HTTP 基础地址保存为 `apiUrl`，同时根据协议生成 `ws://` 或 `wss://` 的 WebSocket 地址，连接 `/start?token=...` 控制入口。连接建立后，前端立即发送订阅消息，并调用统计接口补齐累计拦截次数等低频数据。

为了避免用户在未连接后端时进入控制台，前端路由需要具备访问守卫。`router.beforeEach` 通过 `Pinia Store` 中的 `isConnected` 判断连接状态，当目标页面不是登录页且 WebSocket 未连接时重定向到首页。该需求保证控制台页面始终有明确的数据来源，也避免用户误以为代理核心已经在线。

3.3.2 概览仪表盘

概览仪表盘承担系统首屏观测的作用，用户进入控制台后应能先看到代理运行的大致状态。页面需要展示总上传、总下载、实时上传速率、实时下载速率、安全策略摘要、路由策略摘要、用户流量摘要以及活动连接简表等信息。

3.3.3 连接与历史连接管理

连接管理页需要展示当前活动连接的完整列表，并保留父子连接关系。对于 CONNECT 隧道，父连接表示隧道本身；在 MITM 模式下，代理从隧道中解析出的明文 HTTP 请求应作为子连接展示。对于普通 HTTP 请求，则可以直接作为独立连接展示。

3.3.4 日志管理

日志页面需要接收后端批量推送的 `log_batch` 消息，并提供等级筛选、分类筛选、正则搜索、清空以及自动滚动开关等操作。日志分类应覆盖路由匹配、访问控制、流量统计、请求响应、WebSocket、MITM/TLS、网络错误、系统和通用等类别，使用户能够根据问题类型快速缩小排查范围。

3.3.5 MITM 抓包管理

MITM 页面面向授权调试场景，需要展示请求响应交换记录。列表字段包括交换编号、会话编号、父隧道编号、方法、Host、URL、状态码、耗时、请求大小和响应大小；详情区域需要展示请求头、响应头、请求体上传状态、响应体上传状态、对象 Key、Content-Type 和异常信息。用户可以按会话 ID、父会话 ID、Host、URL 和方法筛选，也可以按状态码、耗时和大小排序。

抓包页面不能直接承载大 Body 内容，因此下载需求通过 `/api/storage/download` 完成。前端根据抓包记录中的对象 Key 构造下载请求，由后端返回代理下载响应或对象存储访问地址。页面还需要提供清空前端缓存的操作，避免长时间调试后浏览器端保存过多抓包记录。

3.3.6 路由、安全、存储和高级配置

配置类页面主要承担系统运行过程中的干预和管理职责。路由配置页需要支持代理节点和路由规则的增删改查，规则类型包括域名后缀、域名关键词和 IP 等，规则命中后的目标动作可以是直连，也可以是转发到某个二级代理节点。安全策略页则需要管理访问控制规则和来源 IP 封禁规则，同时展示实时拦截日志。用户在用户监控页执行的封禁操作，也应同步写入安全策略配置，避免页面操作只停留在临时状态。用户监控页需要按照来源 IP 聚合展示访问情况

3.4 数据与接口需求

3.4.1 配置数据模型

我们设计的配置数据模型以 ServerConfig 为核心。字段设计需要保持 JSON 可序列化。后续如果增加新字段，也应提供合理默认值，保证旧配置文件在升级后仍能被正常读取。

表 3-2 配置数据模型需求表

数据结构	主要字段与需求
ServerConfig	保存端口、MITM、PublicIPs、路由、访问控制、MinIO 和高级行为开关，是配置接口的顶层模型。
ProxyNode	保存二级 HTTP 代理节点名称和 URL，供路由规则通过 Action 引用。
RouteRule	保存规则编号、匹配类型、匹配值、目标动作、启用状态和备注。
AccessRule	保存目标访问控制规则，支持域名后缀、域名关键词和 IP。
UserBlockRule	保存来源 IP 封禁规则，支持逗号分隔的多个 IP 和独立启停。
MinioConfig	保存对象存储 endpoint、publicEndpoint、凭证、bucket、useSSL 和 enabled。

3.4.2 WebSocket 消息模型

WebSocket 消息模型采用主题订阅方式。前端连接后发送 subscribe 消息，其中 topics 数组声明需要接收的主题，后端根据订阅状态推送对应数据。WebSocket 作为双向通信协议，适合承载代理运行期持续变化的数据[2]。系统需要支持的主要消息如表 3-3 所示。

表 3-3 WebSocket 消息需求表

消息类型	数据内容
traffic	推送每秒上传速率、下载速率、累计上传和累计下载。
connections	推送连接快照、连接总数、连接状态和实时流量引用值。
log_batch	批量推送运行日志，并按订阅的日志等级过滤。
mitm_exchange_batch	批量推送 MITM 请求响应交换记录。
intercept_log_batch	批量推送访问控制拦截日志和累计拦截次数。
user_traffic	推送来源 IP、在线状态、总流量和 Host 明细流量。

3.4.3 存储数据模型

存储数据模型主要围绕 MITM 抓包记录设计。HttpExchange 用于表示一次明文请求和响应交换，其中包含交

换 ID、子会话 ID、父隧道 ID、时间戳、请求快照、响应快照、耗时和错误字段。RequestSnapshot 保存请求方法、URL、Host、Header、总大小以及请求体对象信息；ResponseSnapshot 保存状态码、状态文本、Header、总大小以及响应体对象信息。这些字段与前端 MITM 页面中的列表展示和详情查看内容基本对应。

BodyCapture 用于描述请求体或响应体在对象存储中的状态，主要包括对象 Key、实际大小、是否上传成功、Content-Type 和错误信息。系统并不要求 WebSocket 直接传输正文内容，而是通过对象 Key 将前端展示和正文下载分离开来。这样既能保留抓包分析所需的数据入口，也能减少代理进程和浏览器端在处理大请求体或大响应体时的内存占用。

3.5 可行性分析

3.5.1 技术可行性

经过我们的仔细分析，从后端实现角度看，Go 标准库已经提供 HTTP 服务端、HTTP 客户端、TLS、TCP 连接处理和并发控制等能力，能够支撑代理监听、出站请求、CONNECT 隧道和连接统计等需求[3]。从前端实现角度看，Vue 3 的响应式机制和组件化模型适合构建多页面控制台[4]。Pinia 可以集中保存 WebSocket 状态、连接数据、日志、抓包记录和配置内容，Vue Router 则能够组织登录页与控制台内部的多个子页面。

3.5.2 经济可行性

系统依赖的 Go、Vue、Vite、Pinia、Vue Router、Chart.js、TanStack Virtual、Gorilla WebSocket 和 MinIO Go Client 均可以基于开源生态完成开发。部署层面，系统可以采用一个后端代理进程加一个 MinIO 服务的方式运行，前端静态资源可嵌入后端程序，不需要额外购买商业代理平台或专用可视化工具。

对于毕业设计而言，系统运行主要面向本地或实验环境，测试目标服务、代理端口、控制服务和对象存储均可在普通开发机器或容器环境中搭建。系统的经济成本主要是开发、测试和少量运行资源成本，不涉及额外商业授权，因此具有较好的经济可行性。

3.5.3 操作可行性

操作可行性主要体现在系统降低了用户直接接触配置文件和命令行输出的频率。用户通过登录页连接控制服务后，可以从侧边栏进入概览、连接、历史连接、日志、MITM、路由配置、安全策略、用户监控、存储配置和高级设置等页面。系统部署后，控制服务同时提供静态前端资源、REST 配置接口和 WebSocket 实时推送入口。

4 系统总体设计

4.1 系统总体架构设计

4.1.1 整体架构

我们在该系统设计中围绕代理转发、实时观测、策略配置和抓包留存等需求展开设计，整体采用“代理核心 + 控制服务 + 对象存储 + 前端控制台”的分层架构。

表 4-1 系统总体组成与职责划分

组成部分	主要代码位置	设计职责
代理接入层	main.go、mproxy/http.go、mproxy/https.go	监听 HTTP 代理端口和 HTTPS 代理端口，区分普通 HTTP 请求、CONNECT 隧道和 MITM 明文请求。

代理核心层	mproxy/core_proxy.go、mproxy /ctxt.go、mproxy/hooks.go	维护请求上下文、责任链、出站转发、 连接生命周期和异常处理。
控制服务层	proxysocket/proxy.go、proxysocket /hub.go	提供配置 API、统计 API、下载 API、 WebSocket 订阅推送和嵌入式静态资源 服务。
对象存储层	myminio/config.go、myminio /minioUpload.go、myminio /storage_api.go	保存 MITM 请求体和响应体，生成对象 下载地址或通过控制服务转发下载。
前端控制台	proxyui/src/router、proxyui /src/stores、proxyui/src/views	展示概览、连接、日志、抓包、路由、 安全、存储和高级配置等页面。

4.1.2 前后端协作流程

系统前后端协作采用 REST 接口和 WebSocket 两类通道。配置读取、配置保存和对象下载这类调用频率较低、请求响应边界较清楚的操作，主要通过 REST 完成。例如，前端通过 /api/config 获取或保存 ServerConfig，通过 /api/stats 获取进程级拦截统计，通过 /api/storage/download 获取抓包正文的下载信息。全局流量、活动连接、运行日志、MITM 交换记录、拦截日志和用户流量快照等连续变化的数据，则交由 WebSocket 推送。WebSocket 适合浏览器与服务端之间的双向通信[2]，因此这种划分可以减少前端轮询，也便于后端按照订阅主题向不同页面推送所需数据。

4.1.3 运行时数据流

客户端在浏览器、操作系统或接口测试工具中配置代理地址后，请求会进入 HTTP 或 HTTPS 代理监听端口。ServeHTTP 根据请求方法完成第一次分发：非 CONNECT 请求进入 MyHttpHandle，CONNECT 请求进入 MyHttpsHandle。普通 HTTP 请求需要符合代理请求格式，即目标 URL 应为绝对地址；CONNECT 请求则通过 Hijacker 接管底层连接，再由 CONNECT 处理链决定本次请求采用透传、拒绝、HTTPS-MITM 还是 HTTP-MITM。

4.2 后端模块划分

后端模块划分遵循高内聚、低耦合的思路。

表 4-2 后端模块划分表

模块	核心对象	主要职责	对应需求
代理核心模块	CoreHttpServer、 Pcontext、ReqHandler、 RespHandler、HttpsHandler	统一入口分发、上下文传递、 责任链执行、HTTP 转发和 CONNECT 连接接管。	F-01、N-01
配置管理模块	ConfigManager、 ServerConfig	加载默认配置、读写 JSON 配 置、并发安全访问、持久化和 热更新触发。	F-06、N-03

路由模块	Router、OutboundDialer、DirectDialer、HttpProxyDialer	按规则选择直连或二级代理节点，为普通请求、MITM 请求和 CONNECT 隧道提供统一出站能力。	F-03
访问控制模块	AccessController、AccessRule、UserBlockRule	拦截目标域名或 IP，封禁来源 IP，生成拦截日志并关闭命中连接。	F-04、N-04
流量与连接模块	TrafficCounter、ConnectionInfo、GlobalUserMonitor	统计全局、连接和用户维度流量，维护活动连接和历史连接快照。	F-02、F-07
抓包与存储模块	ExchangeCapture、HttpExchange、BodyCapture、MinIO Client	生成 MITM 请求响应元数据，流式保存请求体和响应体，并提供下载入口。	F-05
控制服务模块	WebsocketServer、WebSocketHub、Subscription	处理控制台登录、REST API、静态资源服务、WebSocket 订阅推送和连接操作命令。	F-02、F-06、F-07

4.2.1 代理核心模块

代理核心模块围绕 CoreHttpServer 展开。使用单例模式全局控制流量，增强了代码的内聚度，已经可维护性。

4.2.2 配置管理模块

配置管理模块负责将系统运行参数从代码中抽离出来。配置管理让前后端交互更加方便直观，一次配置可持久化存储。

4.2.3 路由模块

路由模块用于解决同一代理入口下如何选择不同出站路径的问题。系统通过 ConnectWithReqDial 调用 Router.RouteDial 选择合适的拨号器；对于 MITM 或普通 HTTP 出站请求，系统通过 RouterRoundTripper 使用命中拨号器对应的专属 Transport 发起请求。

4.2.4 访问控制模块

访问控制模块从目标地址和来源客户端两个维度进行设计。通过用户访问，地址访问来完成网络管理。

4.2.5 流量监控与连接管理模块

流量监控模块按照全局、连接和用户三个层级组织数据。流量监控模块是一个很重要的核心模块，用于用户直观看当前网络流量动态。

4.2.6 MITM 抓包与存储模块

MITM 抓包模块只在授权调试场景和配置启用条件下工作。ExchangeCapture 记录一次明文请求的捕获状态，包括开始时间、父隧道会话、请求快照、请求体上传状态、响应体上传状态、错误信息和是否跳过发送。请求经过过滤链后生成 RequestSnapshot，响应体关闭时触发 SendExchange，汇总为 HttpExchange 并写入 GlobalExchangeChan。前端收到的是方法、URL、Host、头部、状态码、耗时、大小和对象 Key 等元数据，而不是完整正文。

正文存储由 MinIO 客户端模块承担。MinIO 提供兼容 S3 的对象存储能力[5]，系统通过 BodyCapture 记录对象 Key、实际大小、上传状态、Content-Type 和错误信息。对于已知 Content-Length 的正文，系统使用管道直接流式上传；对于未知长度或 chunked 数据，系统先写入临时文件，获得实际大小后再上传并清理临时文件。对于 WebSocket、SSE 和其他流式响应，系统会跳过或特殊处理正文捕获，避免为了抓包破坏长连接语义。

4.2.7 WebSocket 控制模块

WebSocket 控制模块承担管理平台的主要入口职责。通过websocket长连接，保证了一次订阅持续发布，且可满足同时向多个订阅者同时发布，增加了系统的扩展性，也提高了通信效率。

4.3 前端模块划分

前端模块围绕“连接控制服务、订阅实时数据、展示业务页面、提交配置变更”四类职责划分。Vue 3 提供组件化和响应式能力[4]，Vue Router 负责页面组织[8]，Pinia 负责全局状态共享[9]。项目没有把所有业务逻辑堆叠在单个页面中，而是将登录、仪表盘布局、概览、连接、日志、抓包、路由配置、安全策略、用户监控、存储配置和高级设置拆分为独立视图，便于维护和扩展。

表 4-3 前端模块划分表

模块	主要文件	设计职责
路由与布局模块	src/router/index.js、src/views/dashboard.vue	维护登录页、控制台父路由和各业务子路由，并通过侧边栏组织页面入口。
状态管理模块	src/stores/websocket.js、src/api/api.js	管理 WebSocket 连接、订阅主题、实时数据缓存、配置加载与保存。
概览展示模块	src/views/Overview.vue	展示全局流量、核心状态、活动连接摘要、路由和访问控制状态。
连接管理模块	src/views/Connections.vue、HistoryConnections.vue	展示活动连接和历史连接，支持筛选、排序、父子连接展示和关闭操作。
日志与抓包模块	src/views/Logs.vue、MITM.vue	展示运行日志、MITM 交换记录、请求响应头部、正文对象下载和异常信息。
	src/views/RouteConfig.vue、	

配置与安全模块	AccessControl.vue、UserMonitoring.vue、StorageConfig.vue、AdvancedConfig.vue	提供路由、安全、用户流量、MinIO 和高级行为开关等配置入口。
---------	---	----------------------------------

4.3.1 路由与布局模块

路由布局方式使用户能够在查看运行状态、调整规则、观察拦截日志和下载抓包正文之间快速切换，也便于后续增加新的运维页面。

4.3.2 状态管理模块

状态管理模块集中在 websocket.js。该 Store 维护 WebSocket 实例、连接状态、订阅配置、API 地址、实时流量历史、活动连接、历史连接、日志、MITM 交换记录、拦截日志、拦截次数、用户流量列表和系统配置。前端页面不直接管理底层 WebSocket 协议细节，而是通过 Store 提供的连接、断开、订阅、清理、关闭连接、加载配置和保存配置等方法完成交互。

消息处理采用类型分发。后端推送 traffic 时，Store 将数据追加到最近 60 秒流量历史；推送 connections 时，Store 更新活动连接并把消失或关闭的连接转入历史连接集合；推送 log_batch、mitm_exchange_batch、intercept_log_batch 和 user_traffic 时，Store 分别更新对应页面所需的数据。模块还设置最大日志数、最大抓包记录数、最大拦截日志数和最大历史连接数，避免长时间运行后前端内存无限增长。

4.3.3 页面展示模块

页面展示模块按照业务目标拆分。拆分为不同的管理界面，提升了可观测度和读性。

4.4 关键数据结构设计

关键数据结构是系统总体设计落到代码层面的核心依据。我们在该系统设计中选择围绕请求上下文、连接记录、配置规则和抓包交换四类结构组织数据：请求上下文解决单次请求在多个模块之间传递状态的问题；连接记录解决前端实时展示和主动关闭的问题；配置规则解决路由、安全和存储等运行参数的持久化问题；抓包交换结构解决 MITM 元数据与大正文分离的问题。

表 4-4 关键数据结构设计表

结构名称	关键字段	设计作用
Pcontext	Req、Resp、RoundTripper、ClientIP、Session、parCtx、TrafficCounter、Dialer、exchangeCapture	贯穿请求处理链，连接请求、响应、路由、统计、抓包和异常处理。
ConnectionInfo	Session、ParentSess、Host、Method、URL、ClientIP、Protocol、Status、UploadRef、DownloadRef、OnClose	描述活动连接和历史连接，支撑前端实时展示、父子关系和主动关闭。

ServerConfig	Port、HTTPSPort、MitmEnabled、RouteEnable、ProxyNodes、Routes、AccessRules、UserBlockRules、MinioConfig、PublicIPs	集中保存系统运行配置，支撑配置持久化、热重载和前端配置页面。
HttpExchange	ID、SessionID、ParentID、Time、Request、Response、Duration、Error	描述一次 MITM 请求响应交换，作为前端抓包详情的数据来源。
BodyCapture	ObjectKey、Size、Uploaded、ContentType、Error	记录请求体或响应体对象存储结果，避免大正文直接进入 WebSocket 消息。

4.4.1 代理上下文 Pcontext

Pcontext 是代理核心中最重要的运行时上下文，用于数据流传递消息，记录当前状态。

4.4.2 连接信息 ConnectionInfo

ConnectionInfo 用于描述一个可以在前端展示和操作的连接会话。其字段包括会话号、父会话号、目标 Host、请求方法、URL、远端地址、客户端 IP、协议类型、开始时间、状态、结束时间、上传下载引用以及关闭回调。

4.5 系统安全设计

代理系统位于客户端与目标服务之间，既具备转发能力，也可能接触敏感的网络行为数据。因此总体设计中需要明确安全边界：控制服务访问需要基本身份校验；MITM 能力必须限定在授权调试环境；出站路由需要避免错误配置造成代理自环；抓包正文和配置文件也应避免暴露给无关用户。我们在该系统设计中当前更适合本地实验、课程演示和受控内网环境，生产级安全能力仍需进一步扩展。

4.5.1 控制服务访问控制

控制服务当前使用 token 参数进行简单校验。用户在登录页填写控制服务地址和密钥，前端连接 /start?token=...，后端 loginHandler 校验通过后才允许升级 WebSocket。该设计实现成本较低，适合本课题的本地调试和课程演示场景，也能避免完全开放的控制台被随意连接。配置接口、统计接口和下载接口则由同一控制服务提供，前端在连接成功后即可完成管理操作。

需要说明的是，当前 token 校验并不等同于完整的生产级权限体系。代码中 WebSocket 升级允许跨源请求，控制服务默认也未提供多用户账号、角色权限、操作审计和强制 HTTPS 传输。因此在生产环境中，应在本设计基础上增加账号体系、细粒度权限、审计日志、CSRF 防护、TLS 终止或反向代理访问控制，避免配置接口和连接控制命令被未授权用户调用。

4.5.2 MITM 使用边界

MITM设置统一启动开关，对于websocket和sse连接MITM 模式会在代理侧终止客户端的 TLS 连接，并读取解密后的 HTTP 明文内容，便于分析流量数据，进一步管理内网，保证内网安全。后续如果系统用于更严格的环境，还应补充抓包保留周期、敏感字段脱敏、下载权限校验和对象存储访问控制等能力。

4.5.3 代理自环防护

代理自环是代理系统部署过程中较容易被忽视的问题，是我们在设计时需要特别注意的，避免流量回环导致服务器带宽耗尽，对于云服务器部署场景，系统会提醒用户应在控制台中配置真实公网 IP，使系统能够识别“通过公网地址访问自身代理端口”的情况，从而减少配置错误导致的循环代理风险。

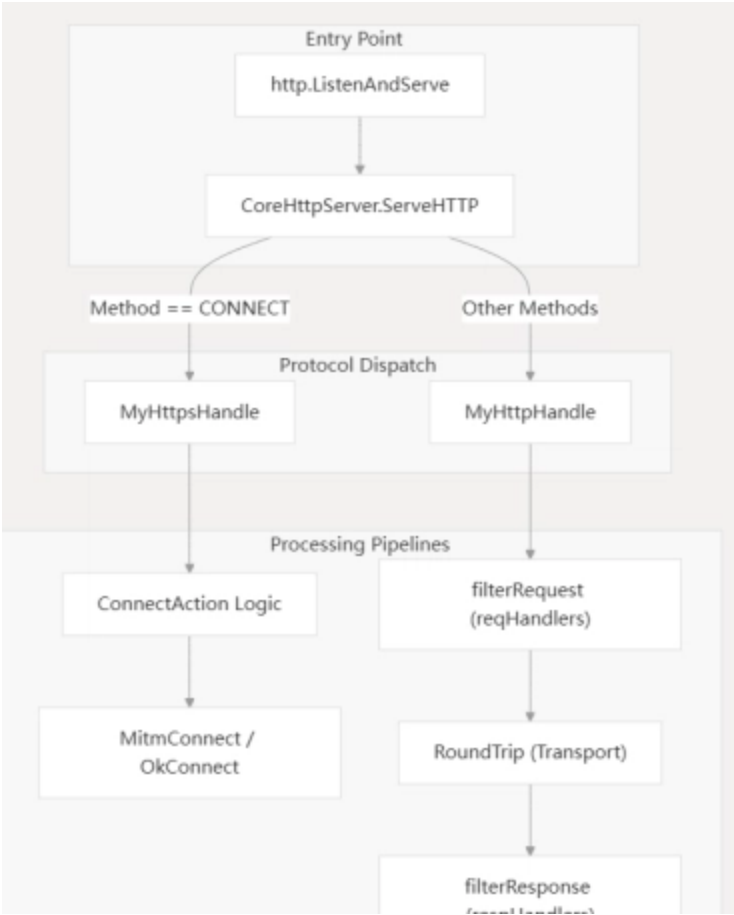
5 系统详细设计与实现

本章在总体设计的基础上，对系统主要功能的代码实现进行说明。系统后端以 Go 程序为运行主体，通过 main.go 完成配置加载、对象存储初始化、路由与访问控制注册、流量监控注册、控制服务启动以及 HTTP/HTTPS 代理监听。代理核心以 CoreHttpServer 为入口，使用 Pcontext 保存单次请求在转发、统计、路由、访问控制和抓包过程中的上下文状态。前端以 Vue 3 页面和 Pinia Store 为核心，通过 REST 接口完成配置读写，并通过 WebSocket 接收流量、连接、日志、MITM 抓包、拦截日志和用户流量等实时数据。

5.1 后端启动流程实现

后端启动入口位于 proxycore/main.go。并将配置对象挂载到 CoreHttpServer 中。后续路由、访问控制、MinIO 初始化和监听端口设置，初始化顺序体现了各模块之间的依赖关系。日志收集器先通过 NewLogCollector 接管代理核心日志输出。InitMinio 根据配置决定是否初始化 MinIO 客户端，即使初始化失败，也不会终止代理主程序。随后，AddRouter 注册路由引擎，AddAccessControl 注册访问控制逻辑，AddTrafficMonitor 注册请求和响应流量统计逻辑。最后，主程序创建 WebsocketServer 并启动控制服务，再创建 HTTP 与 HTTPS 两个代理监听服务。

图5-1 后端启动流程

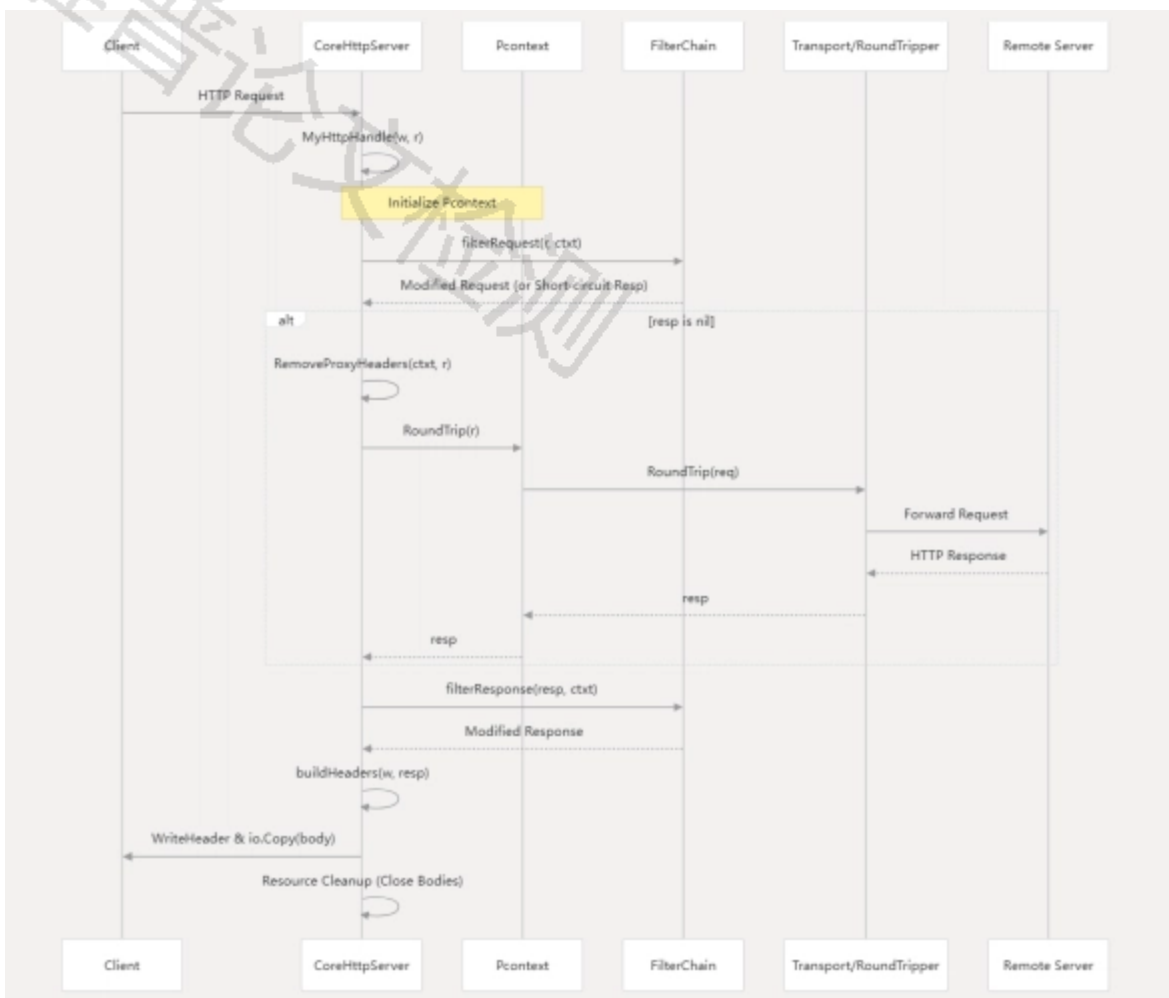


5.2 HTTP 代理实现

5.2.1 普通 HTTP 转发流程

普通 HTTP 请求由 `mproxy/http.go` 中的 `MyHttpHandle` 处理。函数首先判断配置中的 `HttpMitmNoTunnel`：如果该开关开启，则进入基于 TCP 劫持的 HTTP-MITM 引擎；否则使用普通 HTTP 代理流程。普通流程要求请求 URL 是绝对地址。如果请求不是代理格式，系统会交给 `DirectHandler` 返回错误提示，避免代理服务被误当作普通 Web 服务访问。请求进入出站转发前会执行 `filterRequest`。该函数按顺序调用已注册请求处理器，例如访问控制、路由注入和流量统计包装。目标响应返回后，系统先保存原始响应体引用，再执行 `filterResponse`。

图5-2 HTTP转发流程



5.2.2 HTTP-MITM 引擎实现

当 `HttpMitmNoTunnel` 开启时，普通 HTTP 请求会进入 `myHttpHandleWithEngine`。该模式通过 `Hijack` 接管客户端连接，把原本由 `net/http` 管理的连接转为可手工读取和写入的 TCP 连接。函数先为该连接创建一个虚拟父隧道，协议标记为 `HTTP_MUX`，用于保存 `Keep-Alive` 连接级别的流量和生命周期；随后对每个 HTTP 请求创建子

Pcontext 和子连接记录，协议标记为 HTTP-MITM。

图5-3 tls证书生成流程图

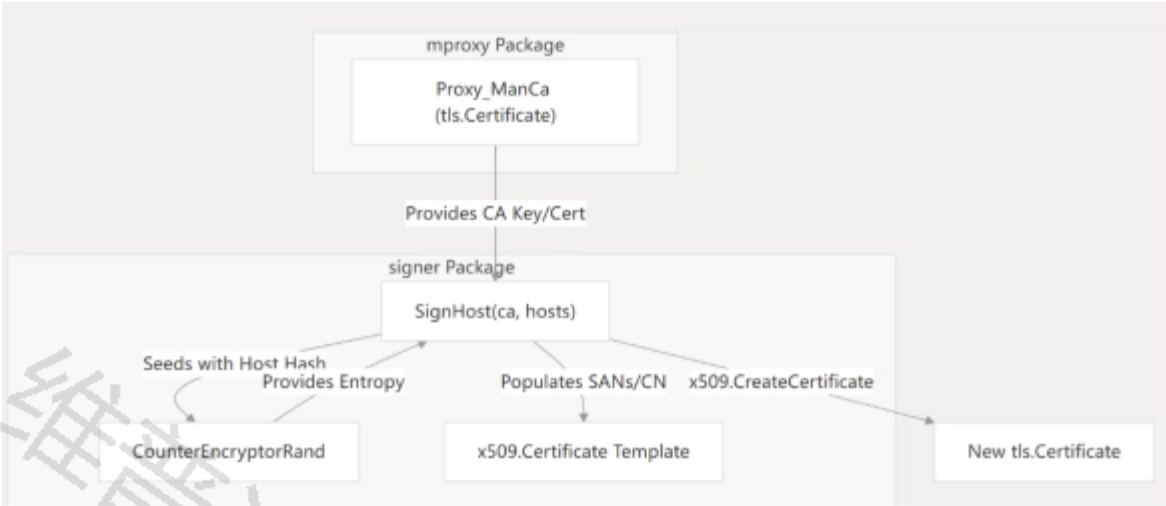
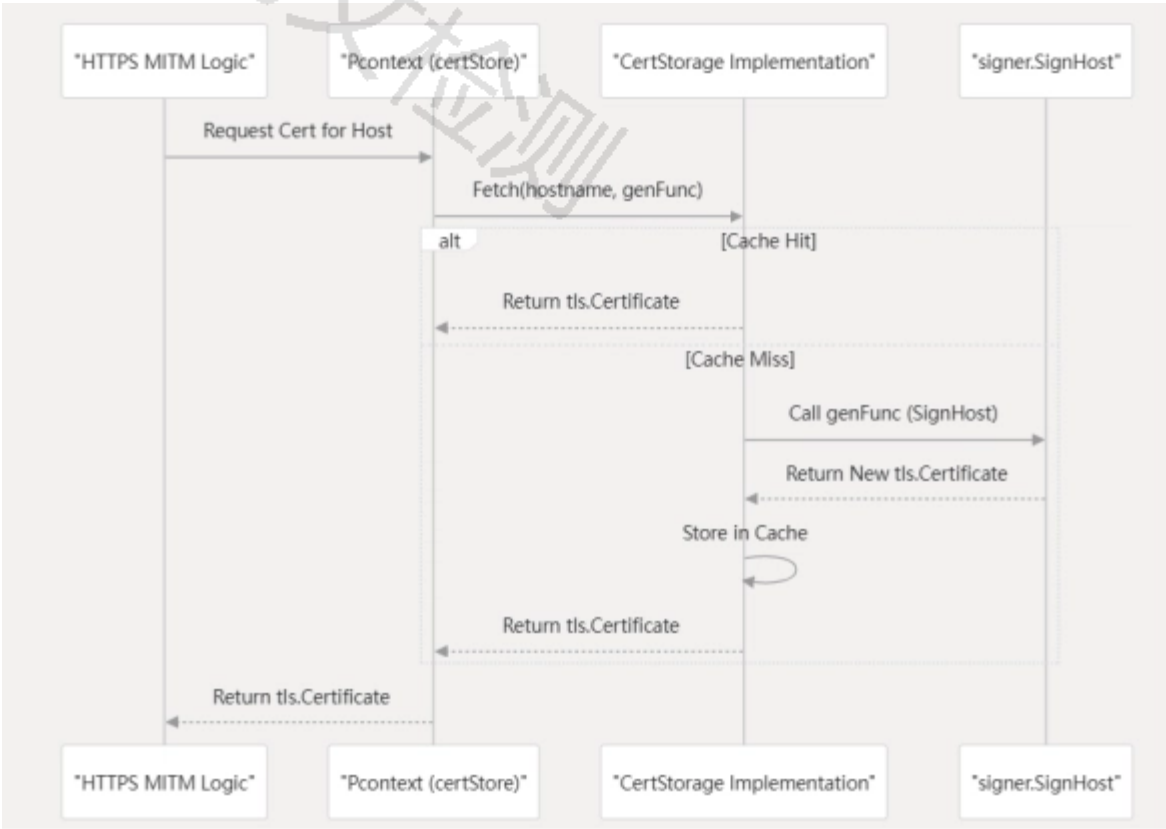


图5-4 tls证书生成时序图



5.2.3 WebSocket 与流式响应处理

WebSocket 和流式响应不能按普通响应体方式处理。MyHttpHandle 在 filterResponse 之后检测响应头是否为 WebSocket 握手。由于响应体经过过滤链包装后可能丢失写能力，系统保留了原始响应体 oriBody，并在 WebSocket 场景中使用 oriBody 作为 io.ReadWriter 与客户端连接进行双向转发。

图5-5 tlswebsockt转发时序图

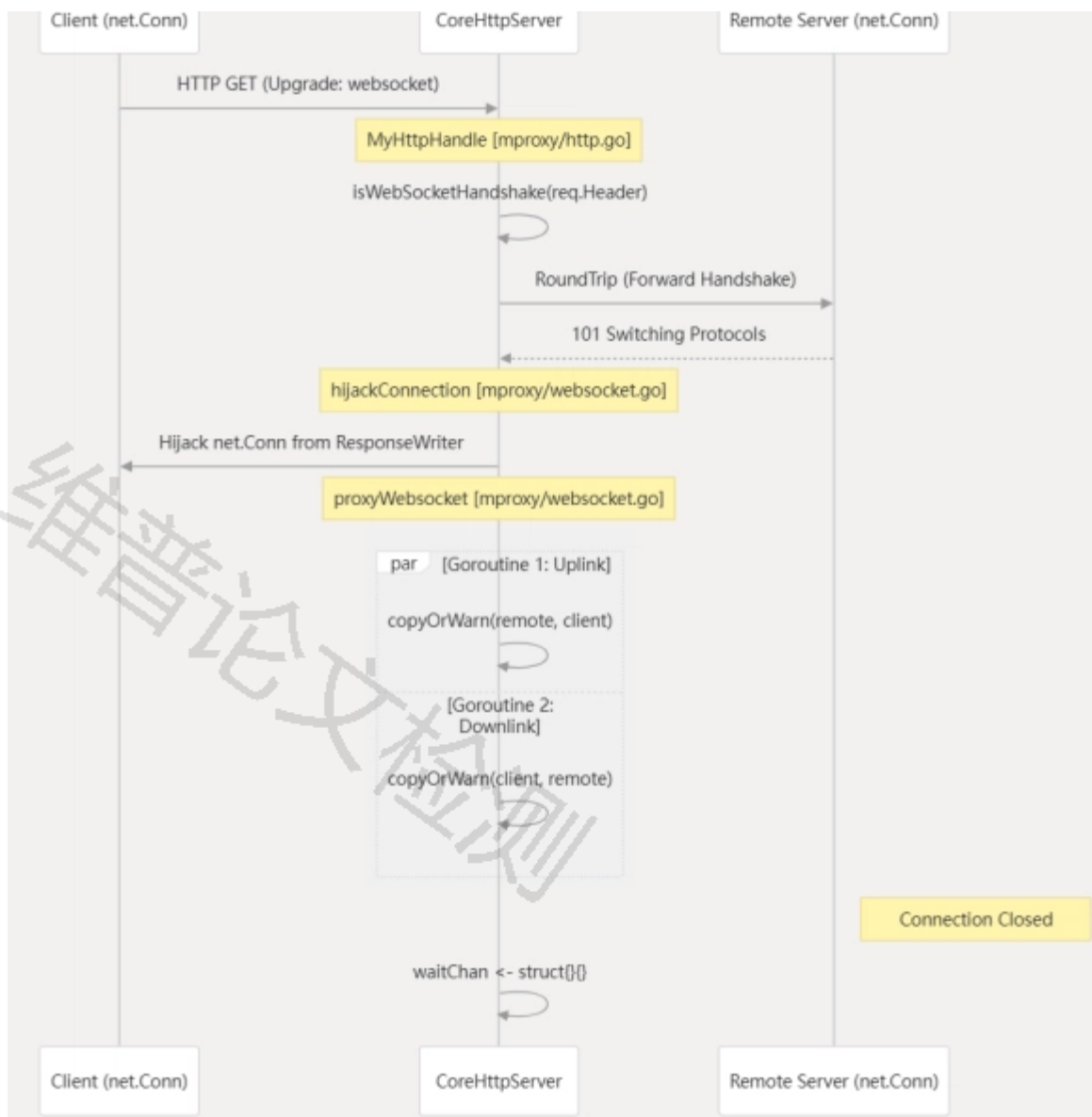
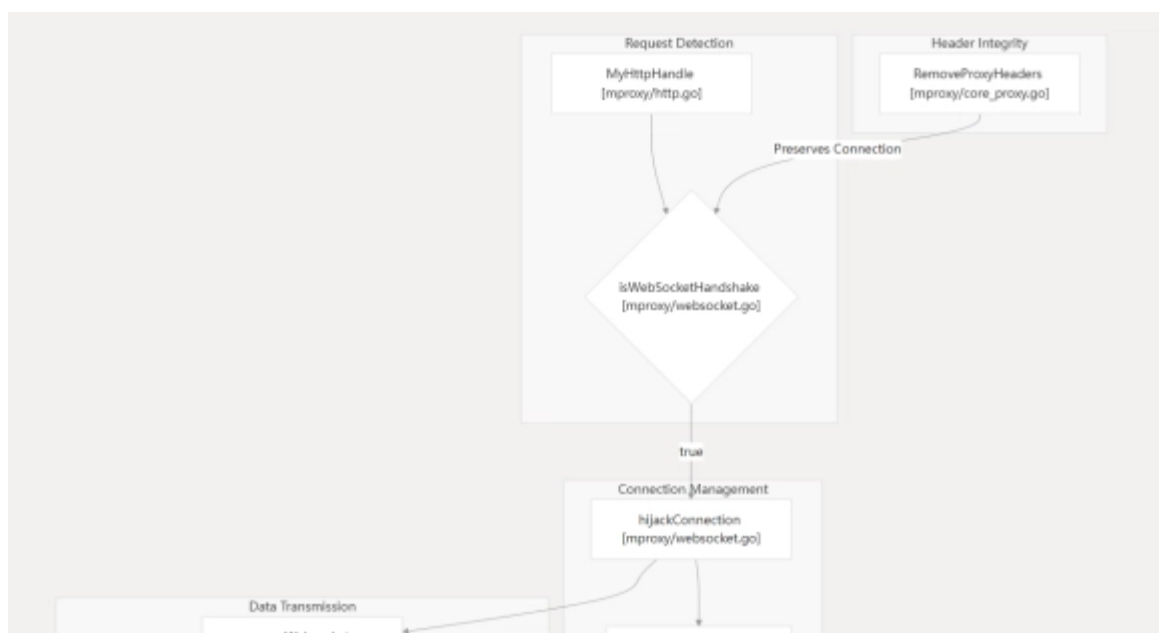


图5-6 tlswebsockt转发流程图





。

5.3 HTTPS 与 CONNECT 实现

5.3.1 CONNECT 隧道模式

HTTPS 代理请求首先以 CONNECT 方法进入 MyHttpsHandle。函数会提取客户端 IP，创建顶层 Pcontext，并通过 Hijack 获取客户端底层连接。随后，系统会在 Connections 中登记一条隧道连接，方法为 CONNECT，协议初始标记为 TUNNEL，同时保存目标 Host、客户端地址、开始时间、流量计数引用和关闭回调。

图5-7 https connect转发流程图

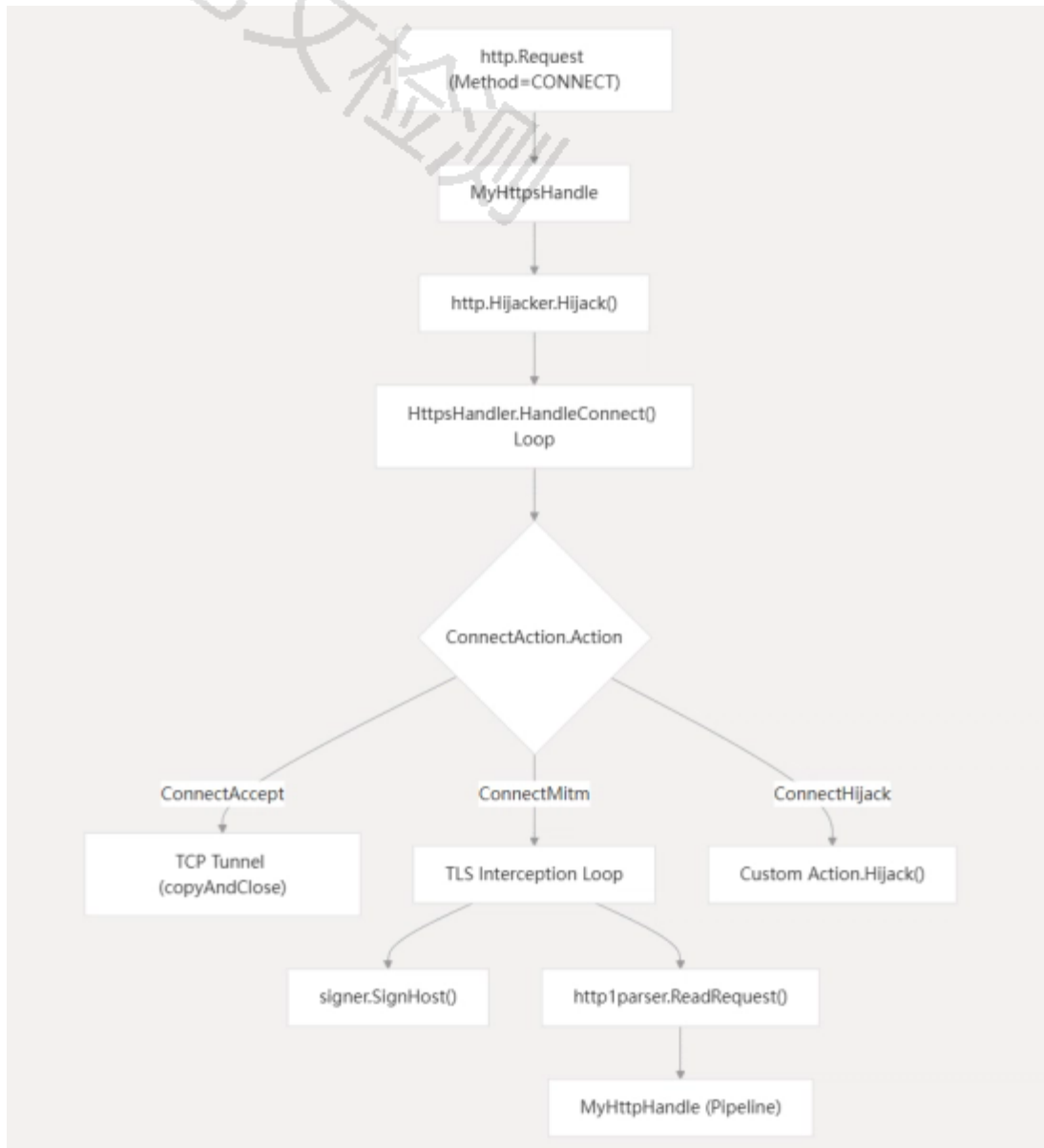
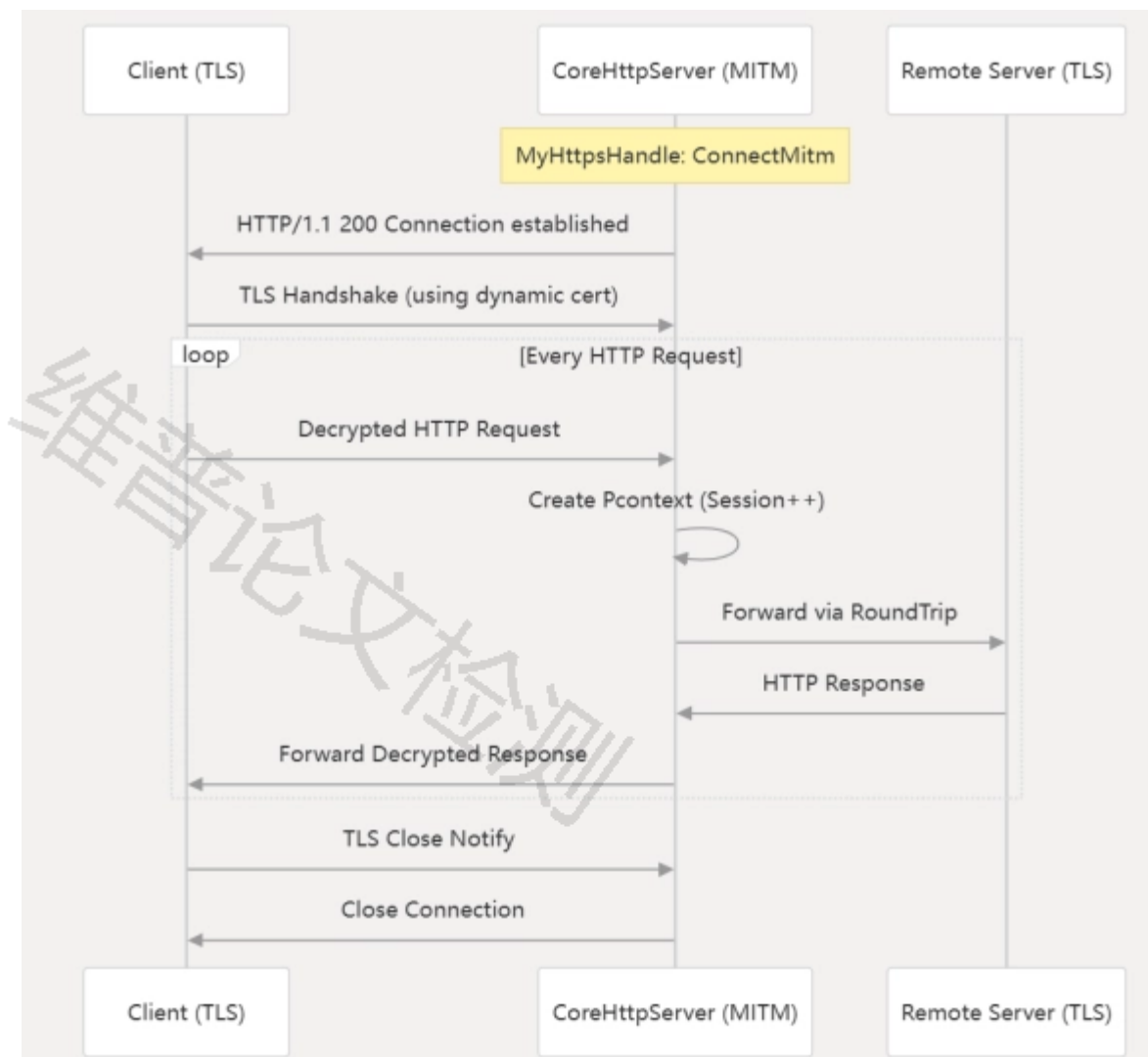


图5-8 https connect转发时序图



5.3.2 MITM 模式

MITM 模式用于授权调试场景下解析 CONNECT 隧道内的 HTTP 请求。进入 ConnectMitm 或 ConnectHTTPMitm 分支后，系统先向客户端写入 200 OK，再通过首字节嗅探判断客户端后续流量是否为 TLS。代码中以 TLS Record Handshake 类型 0x16 作为 TLS 流量识别依据；若识别为 TLS，则执行服务端 TLS 握手；若不是 TLS，则按明文 HTTP 处理。

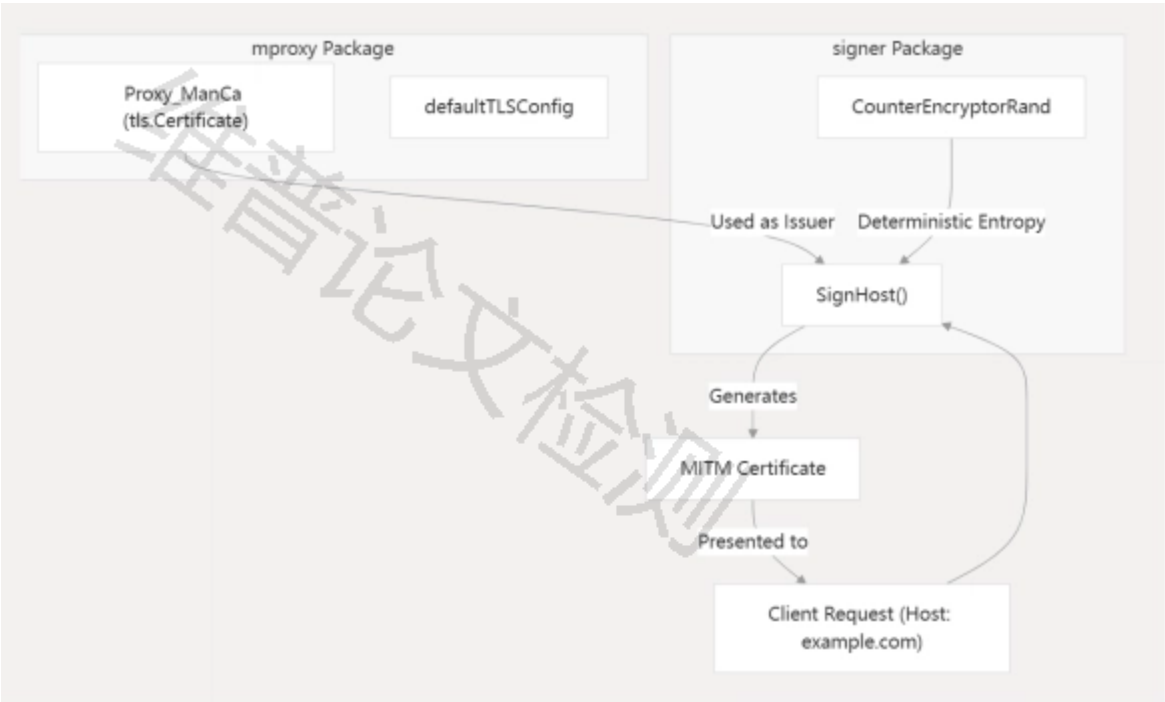
对于 HTTPS-MITM，系统会基于目标 Host 动态生成 TLS 配置，使用 `tls.Server` 与客户端完成握手。握手完成后，隧道内数据变为可解析的明文 HTTP。系统使用 `httpparser.NewRequestReader` 从连接中连续读取 HTTP/1.1 请求，每个请求都创建独立的子 Pcontext，继承父上下文中的客户端 IP、父会话和自定义 RoundTripper，并调用 `StartCapture` 初始化抓包状态。

每个 MITM 子请求的处理流程与 HTTP-MITM 引擎基本一致：登记子连接、执行 `filterRequest`、调用 `CaptureRequest`、放行时执行 `RoundTrip`、执行 `filterResponse`，再手工写回响应头和响应体。若出现本地拦截，

则设置跳过抓包并关闭请求体；若出现 WebSocket 升级，则使用原始响应体进行双向转发并跳过抓包；若响应体被修改，则使用 chunked 编码写回。该复用方式保证 HTTPS-MITM、HTTP-MITM 和普通 HTTP 请求在访问控制、路由、统计和存储行为上保持一致。

父隧道和子请求之间通过 ParentSess 建立关联。父隧道体现底层连接生命周期和隧道总流量，子请求体现每次 HTTP 交换的 URL、方法、状态码、耗时、请求体对象 Key 和响应体对象 Key。前端 MITM 页面展示的是子请求交换记录，连接页面则能呈现隧道与子请求之间的层级关系。

图5-9 MITM 劫持原理



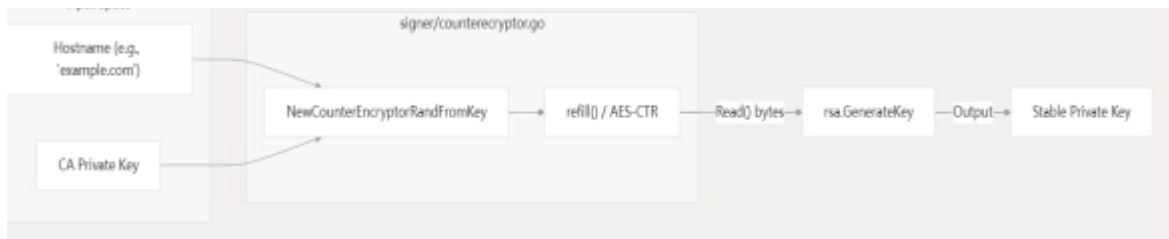
5.3.3 证书签发实现

MITM 证书签发逻辑位于 TLSConfigFromCA。函数接收本地 CA 证书，返回一个根据 Host 生成 TLS 配置的函数。执行时先去除 Host 中的端口，再克隆默认 TLS 配置；若上下文中存在证书缓存 certStore，则通过缓存获取证书，否则调用 signer.SignHost 使用本地 CA 为目标域名动态签发证书。生成的站点证书被加入 TLS 配置后，用于与客户端完成 TLS 握手。

该实现使代理可以针对不同目标 Host 生成匹配域名的证书，从而让客户端在信任本地 CA 的情况下建立 TLS 连接。如果客户端没有信任本地 CA，浏览器或接口测试工具在访问 HTTPS 站点时就会出现证书告警，严重时会导致 TLS 握手失败。因此，论文中需要明确说明 MITM 模式的使用范围：该功能只能用于已经获得授权的开发调试、课程演示和内部测试场景，不能应用在未授权的网络环境中。

证书签发能力和 MITM 开关共同构成系统的安全边界。系统默认仍然可以选择 CONNECT 透传模式，并不要求所有 HTTPS 流量都被解析。只有当用户明确开启 MitmEnabled，并在客户端安装或信任本地 CA 后，系统才会生成明文请求响应记录和正文对象信息。这样的设计既保留了接口调试和抓包分析能力，也没有破坏普通 HTTPS 加密访问场景下应有的基本边界。

图5-9 证书签发

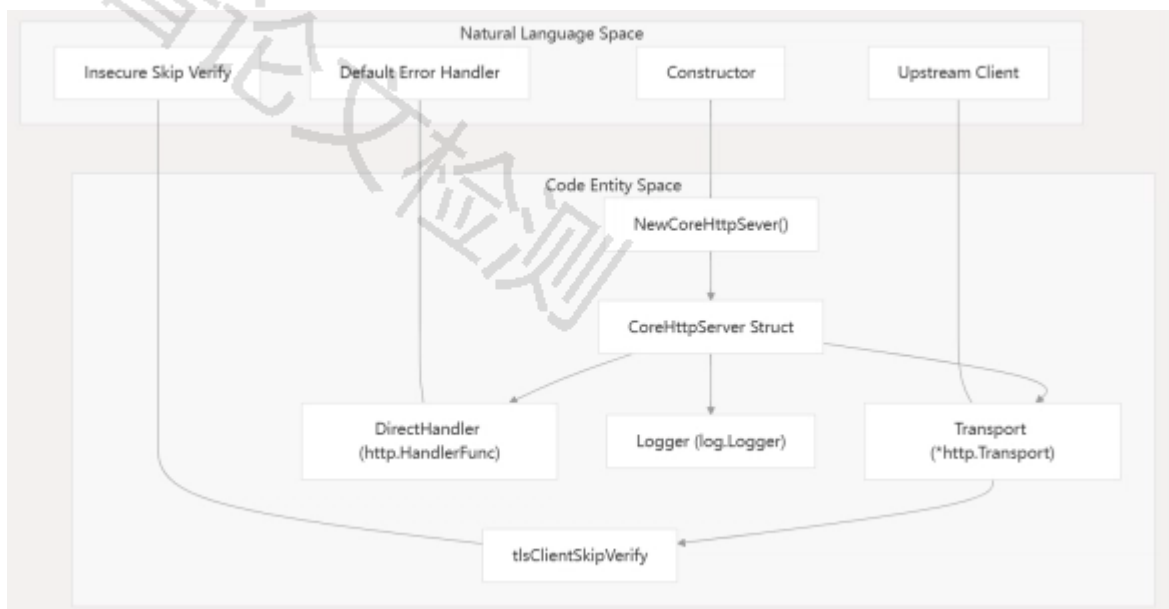


5.4 路由引擎实现

5.4.1 出站拨号器设计

HttpProxyDialer 内部复用 CoreHttpServer.NewConnectDialToProxy，根据代理节点 URL 构造二级代理拨号函数。该函数连接二级代理后写入 CONNECT 请求，并校验返回状态码是否为 200。这样，上层路由模块不需要关心二级代理的握手细节，只需要通过统一接口完成拨号。

图5-10 拨号转发



5.4.2 路由规则匹配

Router 保存 Dialers、Rules 和 Default 三类运行时结构。Rules 中每条规则由匹配条件和目标拨号器名称组成。MatchRoute 会从请求中提取目标信息，按规则顺序依次执行条件判断；如果命中规则且目标节点存在，则返回该节点拨号器；如果目标节点不存在，则记录警告日志并回退到 Direct；如果没有任何规则命中，也返回默认直连拨号器。

路由条件来自配置中的 Routes，支持 DomainSuffix、DomainKeyword 和 IP 三类规则。DomainSuffixRule 用于域名后缀匹配，DomainKeywordRule 用于域名关键词正则匹配，IPRule 用于目标 IP 匹配。路由规则按配置顺序生效，因此用户可以通过调整规则顺序控制优先级。RouteDial 和 RouterRoundTripper 都会输出路由匹配日志，便于用户在日志页解释某个请求为何走直连或二级代理。

该设计覆盖了普通 HTTP、HTTPS 隧道和 MITM 子请求。普通 HTTP 和 MITM 明文请求经过 RoundTrip，因此由 RouterRoundTripper 接管出站；CONNECT 透传不经过 RoundTrip，因此通过 ConnectWithReqDial 调用

RouteDial。两条路径最终复用同一个 MatchRoute，保证路由规则在不同协议入口下行为一致。

5.4.3 路由热重载

配置保存后，控制服务的 handleConfig 会调用 ConfigManager.UpdateConfig 持久化配置；如果 RouteEnable 开启且 Router 存在，则调用 Router.ReloadFromConfig。ReloadFromConfig 的实现采用“锁外构建、锁内替换”的方式：先根据新配置创建 DirectDialer 和所有 HttpProxyDialer，再把启用的规则转换为运行时 RoutingRule，最后在持有写锁的短时间内替换 Dialers、Rules 和 Default。

这种热重载方式降低了运行时阻塞。创建拨号器、解析规则和编译匹配条件可能消耗一定时间，如果在写锁内完成，会阻塞正在匹配路由的请求。当前实现把耗时操作放在锁外，只在替换最终结构时加锁，使配置更新对代理转发的影响更小。

热重载只影响后续请求，不强制打断已经建立的连接。对于长连接或正在传输的大文件，保持原有出站路径可以避免配置保存导致传输中断。若用户希望立即终止旧连接，可通过前端连接管理页关闭全部连接、关闭指定连接或断开某个来源 IP 的连接。

5.4.4 自环检测

代理自环是代理系统中的重要风险。若二级代理节点、路由规则或系统代理环境变量错误指向本机代理端口，请求可能反复回到自身入口，造成连接堆积和资源耗尽。我们在该系统设计中通过 loop_check.go 中的 proxyListenPorts 和 isSelfLoop 判断目标地址是否命中当前 HTTP/HTTPS 代理监听端口以及本机地址或 PublicIPs。

自环检测被布置在多个出站路径中。CoreHttpServer 默认 Transport 的 DialContext 会检查目标地址；DirectDialer 的 Dial 和 Transport DialContext 会检查目标地址；HttpProxyDialer 的 Dial 也会检查目标地址；MyHttpsHandle 中的底层 dial 函数同样会检查目标地址。多处检查可以覆盖普通 HTTP、CONNECT 透传和 MITM 子请求等不同转发路径。

PublicIPs 用于解决公网部署时的识别问题。如果代理部署在云服务器上，请求可能通过公网地址回到自身，而本机网卡地址不一定能覆盖该公网 IP。用户在高级配置中填写 PublicIPs 后，自环检测可以识别“公网地址加代理端口”的错误目标，从而提前阻止代理循环。

5.5 访问控制实现

5.5.1 目标访问控制

访问控制模块位于 mproxy/actions.go，核心对象是 AccessController。NewAccessController 创建控制器后，会分别向请求 Hook 和 CONNECT Hook 注册处理函数。普通 HTTP 请求进入请求 Hook 时，控制器根据请求 URL、Host 和客户端 IP 判断是否需要拦截；CONNECT 请求进入 DoConnectFunc 时，则根据隧道目标 Host 和客户端 IP 判断是否拒绝建立连接。

5.5.2 来源 IP 封禁

来源 IP 封禁规则来自 ServerConfig 中的 UserBlockRules。ReloadFromConfig 会遍历启用规则，按逗号拆分多个 IP，并使用 NormalizeIP 进行标准化处理。合法 IP 会写入 blockedClients 映射，非法值则记录警告日志。请求处理时，控制器会优先检查客户端 IP 是否在封禁集合中；如果命中，则直接拦截该请求。

5.5.3 拦截日志与连接关闭

每次访问控制命中后，系统都会调用 `pushInterceptLog`。该函数会先递增全局 `InterceptCount`，再尝试将 `InterceptLogMessage` 写入 `InterceptLogChan`。写入过程使用非阻塞 `select`，如果通道暂时已满，系统不会阻塞代理请求主流程。拦截日志中记录客户端 IP、目标地址、规则类型、规则值和时间，供前端安全策略页展示。

WebSocket Hub 中的 `StartInterceptLogPusher` 每 500 毫秒批量发送拦截日志，或在批次数量达到阈值时立即推送。推送消息类型为 `intercept_log_batch`，同时携带当前累计拦截次数。前端 `Store` 接收后更新 `interceptLogs` 和 `interceptCount`，概览页与安全策略页都可以同步展示安全策略的运行效果。

连接关闭能力与拦截日志一起构成了访问控制的观测性和可操作性。拦截日志用于说明“为什么被拦截”，连接关闭则说明“策略已经作用到现有连接”。这一点对代理系统比较重要，因为部分代理连接可能持续较长时间，单纯阻止新请求并不能处理已经建立的隧道或长连接。

5.6 流量统计与连接管理实现

5.6.1 HTTP 明文流量统计

HTTP 明文流量统计由 `AddTrafficMonitor` 注入请求 Hook 和响应 Hook。请求阶段先通过 `GetHeaderSize` 估算请求头大小，并写入 `TrafficCounter` 的 `req_header` 和 `req_sum`，同时累加 `GlobalTrafficUp`。

图5-11 流量统计流程图

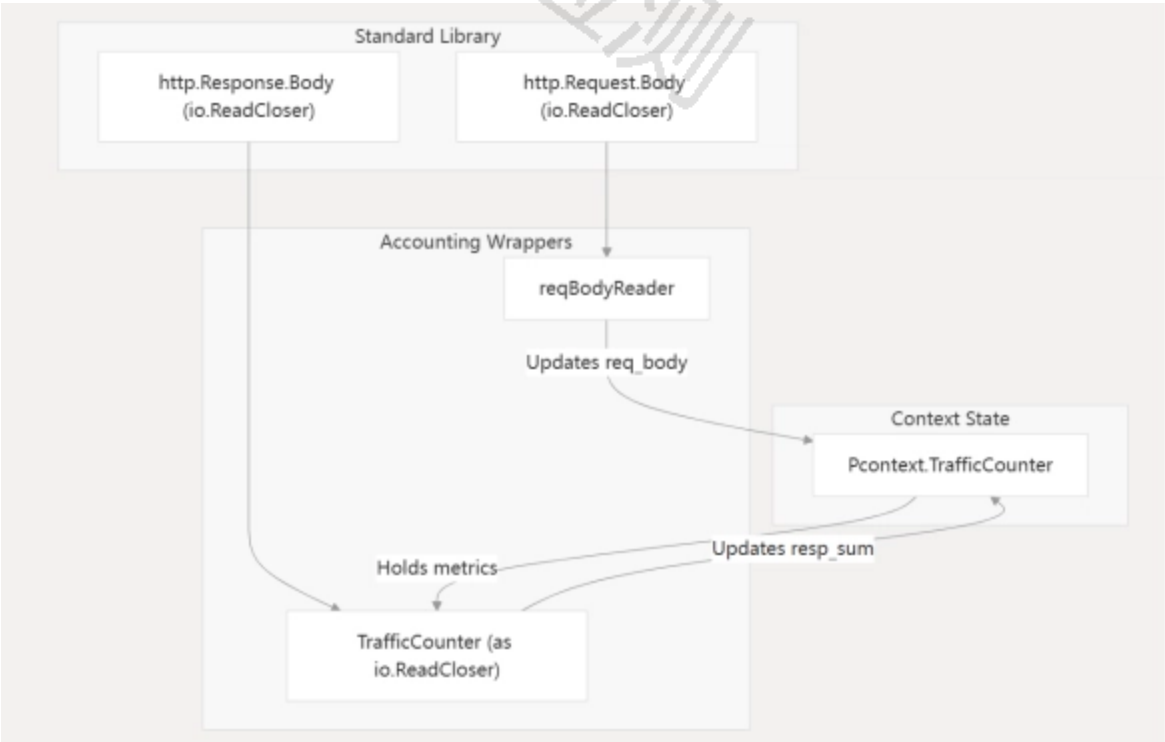
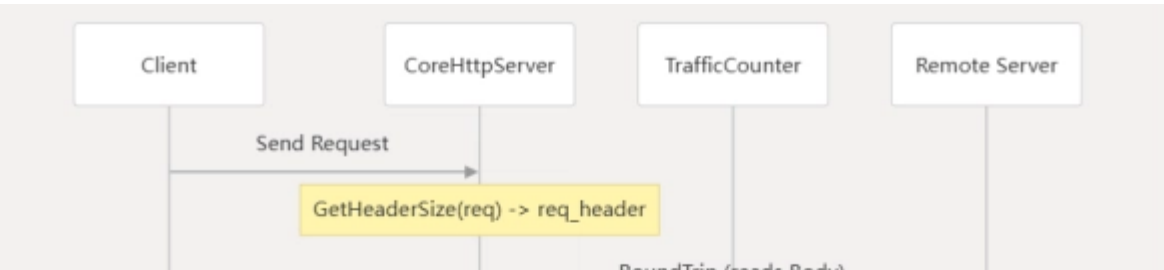
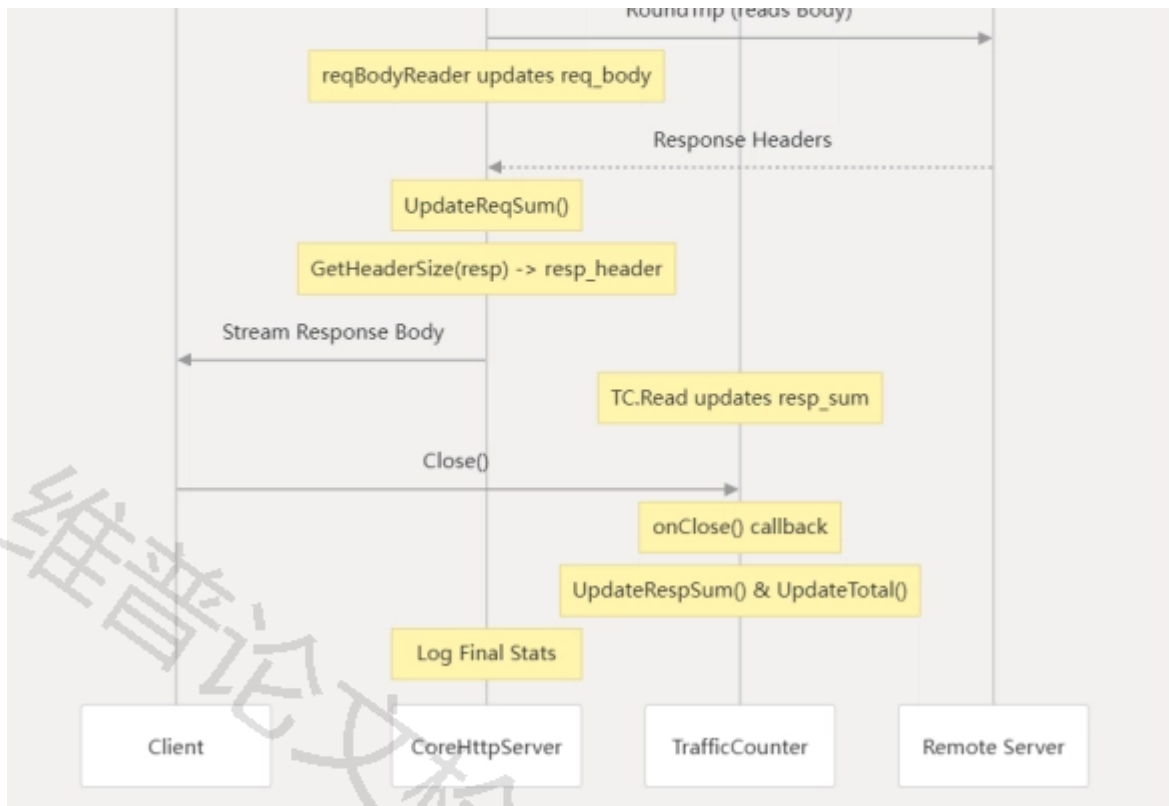


图5-12 流量统计时序图





5.6.2 隧道流量统计

CONNECT 透传模式无法看到 TLS 内部的 HTTP 边界，因此系统按隧道方向统计整体字节流。newTunnelTrafficClient 针对支持半关闭的连接创建包装对象，Read 从客户端读取数据时累计 nread 并累加 GlobalTrafficUp，Write 向客户端写数据时累计 nwrite 并累加 GlobalTrafficDown。对于不支持半关闭的连接，系统使用 tunnelTrafficClientNoClosable 完成类似统计。

隧道流量也会写入用户 Host 统计。MyHttpsHandle 在透传分支中根据 CONNECT 目标提取 Host，并通过 GlobalUserTraffic.GetOrCreateStats 获取当前客户端 IP 和 Host 的统计对象。此后 tunnelTrafficClient 在 Read 和 Write 中分别更新用户上行与下行。这样，即使 HTTPS 透传不能解析明文请求，用户监控页仍能看到某个来源 IP 访问某个 Host 的总体流量。

隧道关闭时，包装连接的 onClose 回调会记录上行、下行和总计，并调用 MarkConnectionClosed。前端连接推送器保留短时间已关闭连接，使历史连接页能够捕获隧道最终状态和流量。该设计使 CONNECT 透传模式在保护明文内容的同时，仍具备连接级观测能力。

5.6.3 连接生命周期管理

连接生命周期由 ConnectionInfo 和 CoreHttpServer.Connections 共同维护。ConnectionInfo 保存 id、parentId、host、method、url、remote、clientId、protocol、startTime、status、endTime、上传下载引用和 OnClose 回调。普通 HTTP 请求、HTTP-MITM 父连接、MITM 子请求、CONNECT 隧道都会登记连接记录，只是协议标签和父子关系不同。

MarkConnectionClosed 不立即删除连接，而是将状态改为 Closed 并记录结束时间。WebSocket 的

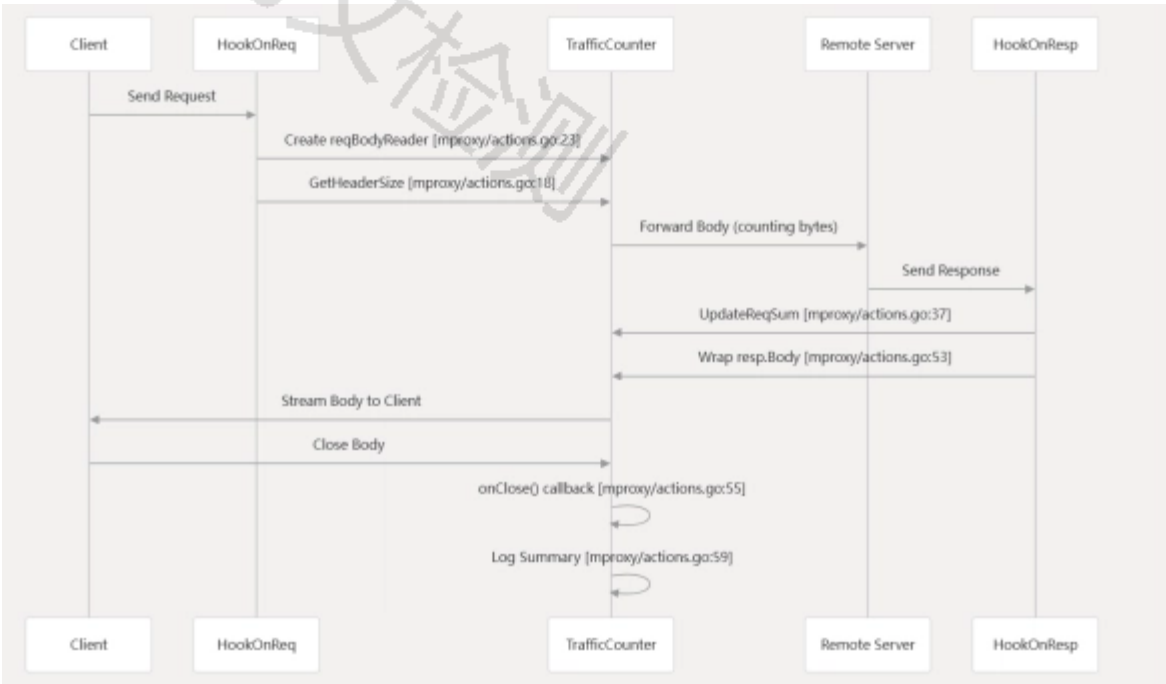
StartConnectionPusher 每 500 毫秒扫描连接表，保留短时间墓碑记录，超过保留时间后再物理删除。推送前，连接按 Active 优先、Session 倒序排序，并限制单次返回数量。这样前端既能看到活跃连接，又能在连接刚结束时捕获关闭状态。

主动关闭由控制服务处理。前端发送 closeAllConnections 时，后端遍历所有连接并调用 OnClose；发送 closeConnection 时，后端先关闭目标连接的子连接，再关闭目标连接本身；发送 closeUserConnections 时，后端按来源 IP 匹配连接并关闭。由于连接记录中保存了关闭回调，前端控制命令可以作用到底层 context 或 net.Conn，实现实际断开。

5.6.4 用户流量聚合

用户流量聚合由 GlobalUserTraffic 维护。该对象在系统中作为全局用户监控器使用，内部数据按两层键组织：第一层是客户端 IP，第二层是目标 Host，每个 IP 与 Host 的组合对应一个 UserHostStats。UserHostStats 中的 Up 和 Down 字段使用 atomic.Int64 保存，适合在请求体和响应体读写过程中进行高频累加，减少普通锁带来的竞争。

图5-13 用户流量时序图



5.7 MITM 抓包与 MinIO 存储实现

5.7.1 抓包快照生成

MITM 抓包数据结构位于 mproxy/mitm_exchange.go。HttpExchange 表示推送给前端的一次请求响应交换，包含交换 ID、子会话 ID、父会话 ID、时间、请求快照、响应快照、耗时和错误信息。RequestSnapshot 保存方法、URL、Host、Header、总大小和请求体对象信息；ResponseSnapshot 保存状态码、状态文本、Header、总大小和响应体对象信息。

每个 MITM 子请求创建 Pcontext 后会调用 StartCapture，初始化 ExchangeCapture 并保存父会话 ID。请求经过 filterRequest 后，系统调用 CaptureRequest 保存请求方法、URL、Host 和头部。响应体关闭时，

respBodyReader 的 onClose 回调会调用 SendExchange, 系统从 TrafficCounter、请求体 BodyCapture 和响应体 BodyCapture 中汇总数据, 生成 HttpExchange 后写入 GlobalExchangeChan。

SendExchange 中设置了多个保护条件: MitmEnabled 关闭时不发送; 捕获状态为空时不发送; skipSend 为 true 时不发送; 已发送过的记录不会重复发送。被本地拦截、WebSocket 升级或不适合捕获的流式场景都会设置跳过标记。这样可以避免前端出现没有真实响应体或不完整的误导性抓包记录。

5.7.2 请求体和响应体捕获

正文捕获由 myminio/minioUpload.go 中的 bodyCaptReader 完成。AddTrafficMonitor 在请求体和响应体的流量统计包装外层, 根据条件再套一层 MinIO 捕获 Reader。只有在 exchangeCapture 存在、MitmEnabled 开启且 MinioClient 可用时才启用捕获; 否则只进行流量统计, 不上传正文。

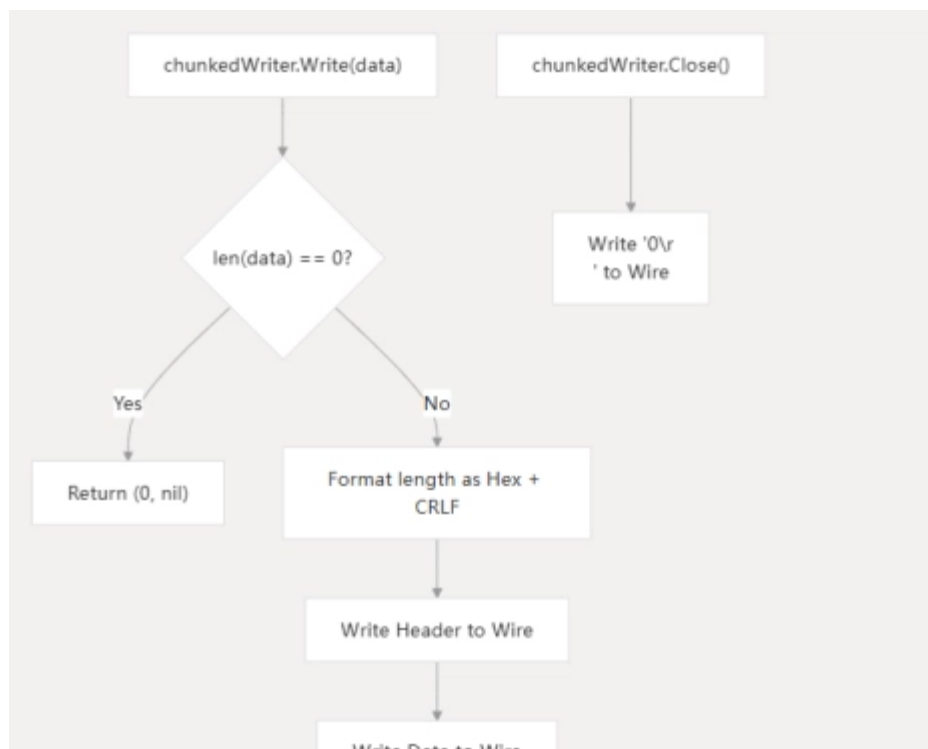
BuildBodyReader 首先判断 MinIO 是否启用以及 Content-Type 是否需要跳过。text/event-stream、websocket、multipart/x-mixed-replace 等流式类型会被跳过, 避免无限流或长连接被错误上传。对于需要捕获的正文, 函数创建 io.Pipe, 并生成 BodyCapture, BodyCapture 中保存对象 Key、Content-Type、上传大小、上传状态和错误信息。随后启动上传协程 uploadToMinIO。

bodyCaptReader 的 Read 方法先从内层 Reader 读取数据, 再把读取到的字节写入 PipeWriter。主转发链路仍然按原节奏读取请求体或响应体, 上传协程则从 PipeReader 中获得同一份数据并写入 MinIO。Close 方法关闭 PipeWriter, 等待上传协程完成, 再关闭内层 Reader。这样正文捕获是转发链路的旁路, 不需要把完整正文一次性读入内存。

5.7.3 大文件与 chunked 上传处理

uploadToMinIO 根据 contentLength 选择不同上传策略。若 Content-Length 已知且大于等于 0, 系统直接调用 PutObjectWithSize, 以指定大小将 PipeReader 中的数据流式上传到 MinIO。

图5-14 chunked流量解析





5.7.4 下载接口实现

对象下载接口由 `myminio/storage_api.go` 中的 `HandleDownload` 提供，并挂载到控制服务的 `/api/storage/download`。前端传入 `key` 参数后，接口首先检查 `Key` 是否为空，并根据对象 `Key` 解析建议文件名。若请求带有 `proxy=1`，接口进入控制服务代理下载分支；否则先检查对象是否存在，再生成浏览器可访问的下载 URL。

下载 URL 的生成优先使用 `PublicClient`。如果用户配置了 `PublicEndpoint`，系统生成对象存储直连的预签名 URL；如果未配置 `PublicEndpoint`，则回退到控制服务自身的代理下载 URL，并附带 `key` 和 `proxy=1` 参数。代理下载分支会调用 `StatObject` 获取对象信息，再调用 `GetObject` 读取对象内容，设置 `Content-Type`、`Content-Disposition`、`Content-Length` 和 `Cache-Control` 后通过 `io.Copy` 写回浏览器。

该接口实现了“实时推送元数据、按需下载正文”的抓包方式。`WebSocket` 消息只携带 `BodyKey`、`BodySize`、`BodyUploaded` 和 `BodyError` 等轻量字段；只有用户在 MITM 页面点击下载时，前端才请求正文。这样既降低了 `WebSocket` 消息体积，也避免浏览器在长时间抓包过程中保存大量二进制内容。

5.8 WebSocket 控制服务实现

5.8.1 连接认证与订阅

`WebSocket` 控制服务由 `WebsocketServer.StartControlServer` 启动。服务在 `/start` 路径挂载登录处理器，`loginHandler` 会读取 URL 中的 `token` 参数，并与服务端 `Secret` 比对。不一致时返回 `401 Unauthorized`，通过后才进入 `handleWebSocket` 完成协议升级。当前实现使用简单 `token` 适合本地实验和毕业设计演示，后续若用于生产环境还需要扩展账号、权限和 `HTTPS` 控制通道。

连接升级后，`handleWebSocket` 创建 `Subscription`，默认日志级别为 `INFO`，并把 `WebSocket` 连接和订阅对象保存到 `Hub` 的 `clients` 中。前端发送 `action` 为 `subscribe` 的消息后，`Hub.updateSubscription` 根据 `topics` 数组更新 `Traffic`、`Connections`、`Logs`、`MitmDetail`、`InterceptLogs` 和 `UserTraffic` 等布尔字段，同时保存 `LogLevel`。

订阅机制使后端可以按页面需要推送数据。概览页通常需要 `traffic` 和 `connections`；日志页需要 `logs`；

MITM 页面需要 `mitm_detail`；安全策略页需要 `intercept_logs`；用户监控页需要 `user_traffic`。当前前端 Store 默认开启部分订阅，并在进入或离开用户监控、安全策略等页面时调整订阅状态，减少不必要的数据传输。

5.8.2 实时数据推送器

WebSocket Hub 启动后会同时启动多个推送器。`StartTrafficPusher` 每秒读取 `GlobalTrafficUp` 和 `GlobalTrafficDown`，计算与上一时刻的差值并归一化为 `bytes/s`，同时推送累计上传和累计下载。`StartConnectionPusher` 每 500 毫秒扫描连接表，生成活动连接和短时间关闭连接的快照，并推送给订阅 `connections` 的客户端。

`StartLogPusher` 从 `LogChan` 中批量收集日志，最多 200 条或每 300 毫秒发送一次。`StartMitmDetailPusher` 负责从 `GlobalExchangeChan` 中收集 `HttpExchange`，当累计达到 100 条，或距离上次发送已超过 500 毫秒时，就将当前批次推送给前端。`StartInterceptLogPusher` 从 `InterceptLogChan` 中收集拦截日志，同样在达到数量阈值或 500 毫秒定时条件后批量发送。`StartUserTrafficPusher` 则每 1.5 秒生成一次用户流量快照，并推送到订阅了用户流量主题的客户端。

这些推送器采用不同频率，主要是因为数据本身的变化特点并不相同。流量速率适合按秒刷新，连接状态和拦截信息变化较快，因此采用 500 毫秒级别的推送；日志和抓包记录可能短时间内大量产生，更适合通过批量机制合并发送；用户流量属于聚合数据，对实时性的要求略低，所以采用 1.5 秒推送。通过这种分频处理，系统能够在保持可观测性的同时，降低前端渲染和网络传输压力。

5.9 前端控制台实现

5.9.1 登录页实现

前端登录页由 `proxyui/src/views/loginView.vue` 实现，核心连接逻辑在 `src/stores/websocket.js`。用户输入控制服务地址和密钥后，`connect` 函数根据 `http` 或 `https` 自动选择 `ws` 或 `wss` 协议，并拼接 `/start?token=...` 作为 WebSocket 地址。连接打开后，Store 将 `isConnected` 设置为 `true`，立即发送 `subscribe` 消息，并通过 `loadStats` 获取累计拦截次数。

WebSocket 关闭后，Store 将连接状态置为 `false`，并在 5 秒后尝试重连。错误事件也会更新连接状态。登录成功后页面进入控制台，连接地址保存在 `apiUrl` 中，后续配置接口和下载接口都以该地址作为基础。该实现使前端不需要写死控制服务地址，适合本地、局域网或远程代理核心切换。

路由守卫位于 `src/router/index.js`。除了登录页以外，访问其他路由时都会检查 Pinia Store 中的 `isConnected`。若未连接 WebSocket，则重定向到登录页。这样可以避免用户直接打开控制台子页面时看到空数据或错误操作入口，也使“先连接代理核心，再进入控制台”的使用流程更加明确。

5.9.2 仪表盘布局实现

`dashboard.vue` 中的 `coreStatusText` 根据 Store 的 `isConnected` 和 `apiUrl` 计算。刷新配置按钮调用 `wsStore.loadConfig`，用于从后端重新拉取当前配置。导航高亮通过当前 `route.path` 判断，安全策略相关子路由统一识别为安全分组。该布局把高频观测页面和低频配置页面集中在同一操作台中，符合系统可视化管理目标。

从页面组织上看，前端没有把所有功能堆叠在一个页面，而是按照业务模块拆分为独立视图。这样既便于用户按任务切换，也便于后续维护。连接、日志和 MITM 页面主要消费 WebSocket 实时数据；路由、安全、存储和高级

设置页面主要消费 REST 配置接口；概览页则汇总多个数据源形成全局状态。

5.9.3 概览页实现

概览页由 Overview.vue 实现，数据主要来自 Store 中的 trafficHistory、connections、interceptCount、userTrafficList 和 config。Store 在收到 traffic 消息时，把速率和累计流量追加到最近 60 秒历史数组；概览页基于该数组展示上传和下载速率曲线。项目依赖中包含 Chart.js，用于绘制实时流量图表。

概览页还展示总上传、总下载、实时上传速率、实时下载速率、安全策略状态、路由策略状态和用户流量摘要。活动连接摘要来自 connections 消息，页面只展示部分连接并提供跳转详细连接页的入口。这样的设计使用户进入控制台第一屏时就能判断代理核心是否在线、当前是否有流量、策略是否启用以及是否存在异常活跃连接。

概览页并不承担所有详细操作，而是作为状态总览入口。用户需要查看单连接细节时进入连接页，需要查看请求响应明细时进入 MITM 页面，需要调整规则时进入路由或安全页面。这种分层设计减少了首页信息过载，也使不同页面职责更加明确。

5.9.4 连接页与历史连接页实现

连接页由 Connections.vue 实现，数据来自 Store 的 connections。后端推送的每条连接包含 id、parentId、host、method、url、remote、clientId、protocol、startTime、status、up 和 down 等字段。前端可以根据 parentId 构建父子关系，将 CONNECT 隧道与 MITM 子请求关联展示。页面支持筛选、排序、展开和关闭连接。

历史连接页由 HistoryConnections.vue 实现。Store 的 handleConnections 会比较上一次活动连接列表和本次推送列表，如果旧连接消失或新状态为 Closed，就将连接快照写入 historyConnections，并用 historyConnectionsSet 去重。历史连接最多保留固定数量，避免长时间运行导致前端内存不断增长。

详细连接和历史连接都适合使用虚拟滚动。项目依赖 @tanstack/vue-virtual，连接、历史连接、日志和 MITM 等大列表页面可以减少实际 DOM 节点数量，提高高频推送场景下的渲染性能。前端关闭连接时，会通过 WebSocket 发送 closeConnection 或 closeAllConnections，后端执行相应 OnClose 回调。

5.9.5 日志页实现

在我们的系统中日志页是系统调试的重要辅助页面。路由配置是否命中、访问控制为何拦截、MinIO 是否初始化失败、WebSocket 是否断开、TLS 握手是否失败等信息都可以通过日志观察。与概览页相比，日志页提供了更细粒度的运行解释能力。

5.9.6 MITM 页面实现

MITM 页面由 MITM.vue 实现，数据来自 Store 的 mitmExchanges。后端推送 mitm_exchange_batch 后，Store 的 handleMITMExchangeBatch 会把每条 HttpExchange 转换为扁平化对象。buildExchangeObject 会提取会话 ID、父会话 ID、时间、耗时、错误、请求方法、URL、Host、请求头、请求大小、响应状态码、响应头、响应大小，以及请求体和响应体的对象存储信息。

5.11 设计模式在系统中的应用

5.11.1 观察者模式（发布-订阅模式）的应用

观察者模式我们在该系统设计中主要体现为 WebSocket 发布-订阅结构。后端的全局流量计数器、连接

表、日志通道、MITM 交换通道、拦截日志通道和用户流量监控器都是被观察的数据源；WebSocket Hub 维护客户端连接和 Subscription，根据客户端订阅主题决定是否推送对应数据。这种设计适合代理系统的实时监控场景。流量、连接和日志都属于持续变化数据，如果前端采用轮询会造成额外请求开销；发布-订阅方式可以在数据产生后按主题推送给需要的页面。新增主题时，只需扩展 Hub、Store 和相应页面，不需要改动代理转发主流程。

5.11.2 责任链模式的应用

责任链模式贯穿后端代理处理过程。CoreHttpServer 中保存 reqHandlers、respHandlers 和 httpsHandlers 三类处理器数组，filterRequest、filterResponse 和 CONNECT 策略选择会按注册顺序执行这些处理器。HookOnReq、HookOnResp 和 DoConnectFunc 提供了把普通函数适配为处理器的方式。

请求链负责访问控制、路由 RoundTripper 注入、请求头统计、请求体包装和请求快照采集；响应链负责响应头统计、响应体包装、流量收尾和抓包记录发送；CONNECT 链负责在隧道透传、HTTPS-MITM、HTTP-MITM 和拒绝连接之间选择动作。每个处理器只处理自身职责，并可以通过返回本地响应或连接动作提前终止后续流程。

责任链模式使系统具备较好的扩展性。若后续需要增加基于 Header 的访问规则、响应内容脱敏、请求重写或审计记录，可以作为新的 Hook 插入到已有链路中，而不必重写 MyHttpHandle 或 MyHttpsHandle。对于毕业设计而言，该模式也使详细设计能够清楚对应到代码结构。

5.11.4 适配器模式的应用

适配器模式主要用于统一不同实现形态。OutboundDialer 将直连拨号和二级 HTTP 代理拨号适配为同一接口，上层路由模块只需要调用 Dial、Name 和 GetTransport，而不需要关心底层是否需要先连接二级代理并发送 CONNECT 请求。RouterRoundTripper 则把路由选择能力接入到 http.RoundTripper 风格的请求发送流程中，使普通 HTTP 请求和 MITM 子请求都能复用同一套路由规则。

Body 处理同样体现了适配器思想。reqBodyReader 和 respBodyReader 将原始 io.ReadCloser 包装为具备流量统计能力的 Reader；bodyCaptReader 在外层继续包装，使同一份数据既能正常转发，又能写入 MinIO；tunnelTrafficClient 则把 net.Conn 包装为带有上行、下行计数和关闭回调的连接。上层转发代码仍然面对 io.Reader、io.ReadCloser 或 net.Conn，不需要了解底层具体叠加了多少层包装。

前端 Store 也承担了一部分数据适配功能。后端 HttpExchange 是嵌套结构，Store 中的 buildExchangeObject 会将其转换为页面更容易筛选和排序的扁平对象；连接推送中的 parentId、up、down 等字段，也会被 Store 整理为页面可以直接消费的数据。通过这些适配层，系统把复杂的底层实现转换为统一、稳定的上层接口，提高了后续维护和扩展的便利性。

6 系统测试与结果分析

本章围绕可视化网络代理管理系统的功能正确性、并发稳定性和资源占用情况进行测试设计与结果分析。测试对象包括 Go 后端代理核心、HTTP/HTTPS 测试后端、MinIO 流式存储链路、WebSocket 实时推送以及 Vue 前端管理页面。由于系统同时支持普通 HTTP 代理、HTTPS CONNECT 隧道、MITM 明文解析、KnownSize 传输和 Chunked 传输，测试章节按照“功能验证、压力基准、资源剖析、问题归纳”的顺序展开，使第 3 章提出的功能需求和非功能需求都能对应到具体测试方法。

本章测试以项目中已有测试代码和测试命令为基础，不重新设计一套脱离实现的测试流程。后端测试服务由

test/backend_server.go 提供，HTTP 服务监听 9011 端口，HTTPS 服务监听 9012 端口，并提供 /test/upload、/test/download 和 /test/download/chunked 等接口。压力测试主要依赖 test/gotest/fetch_test.go 中的 Go benchmark，测试客户端通过 127.0.0.1:8080 访问代理，并利用 net/http 的 Transport、连接池和 RunParallel 并发模型持续产生上传、下载和隧道访问流量[3]。

6.1 测试环境与测试方法

6.1.1 测试环境

采用 Windows 桌面环境进行基准测试

表 6-1 测试主机与运行环境

项目	配置或说明
处理器	Intel(R) Core(TM) i7-9750H CPU @ 2.60GHz
核心与线程	6 个物理核心，12 个逻辑处理器
运行频率	截图记录时约 4.00GHz，基准频率约 2.59GHz
内存	15.9GB，总占用约 12.0GB
磁盘	系统盘为 SSD，另有 HDD 数据盘；测试文件从本地磁盘读取
网络	本机回环地址用于代理、后端服务和 pprof 采集
代理服务	HTTP 代理端口 8080，控制与 pprof 调试端口 6060
测试后端	HTTP 服务 9011，HTTPS 服务 9012
对象存储	MinIO 用于 MITM 请求体与响应体对象留存
测试文件	统一使用 5MB 中型文件 huge_5m.bin，避免不同文件大小导致结果不可比

测试数据由 test/gendata/generate.go 生成，其中 huge_5m.bin 的大小为 5×1024×1024 字节。正式压力测试统一使用该 5MB 文件作为上传体或下载体，原因有三点：第一，5MB 文件能够覆盖请求体、响应体、对象存储上传和连接统计等完整链路；第二，其规模大于小型接口请求，能够暴露 Chunked 临时文件、MinIO Pipe 上传和响应写回过程中的资源问题；第三，相比 100MB 或 200MB 文件，5MB 文件更适合在普通笔记本测试环境中多轮复现实验。

6.1.2 测试方法

测试方法主要分为单元测试、功能测试、压力基准测试和 pprof 性能分析四类。

表 6-2 压力测试命令分组

测试组	命令或采集方式
未开启 MITM，HTTP 压力	go test -bench=Benchmark_Stress_HTTP_* -benchtime=10s -run='^\$' -v -cpu 6

未开启 MITM, HTTPS 压力	go test -bench=Benchmark_Stress_HTTPS_.* benchtime=10s -run=^\$ -v -cpu 6
开启 MITM, HTTP 压力	go test -bench=Benchmark_Stress_HTTP_.* benchtime=10s -run=^\$ -v -cpu 6
开启 MITM, HTTPS 压力	go test -bench=Benchmark_Stress_HTTPS_.* benchtime=10s -run=^\$ -v -cpu 6
KnownSize 与 Chunked 对比	go test -bench=".*(Chunked KnownSize)\$" benchtime=10s -run=^\$ -v -cpu 6
大流量压测	g o t e s t bench=Benchmark_Stress_HTTP_Upload_KnownSize benchtime=20s -run=^\$ -v -cpu 8
堆内存采集	go tool pprof -http=:8081 http://localhost:6060 /debug/pprof/heap
CPU 采集	go tool pprof -http=:8083 "http://localhost:6060 /debug/pprof/profile?seconds=15"

表 6-2 中的命令用于规定测试复现口径。执行时应先启动测试后端、代理核心和 MinIO，再进入 proxycore /test/gotest 目录运行对应 benchmark。pprof 命令需要在压力测试运行期间另开终端执行，以保证采集到代理处于负载状态下的 CPU 调用栈和堆内存分配情况。

6.2 单元测试设计

6.2.1 HTTP 解析器测试

HTTP 解析器测试主要覆盖 http/parser/request_test.go 和 http/parser/header_test.go。request_test.go 用于验证标准请求、非标准请求以及同一连接中连续多个请求的读取能力，重点对应 HTTP-MITM 和 HTTPS-MITM 模式下从底层连接连续解析 HTTP/1.1 请求的实现需求。header_test.go 则检查头部提取逻辑在空数据、正常数据和非法数据下的行为，保证连接统计和抓包快照能够获得相对稳定的头部字节信息。

图6-1 http解析器工作流程

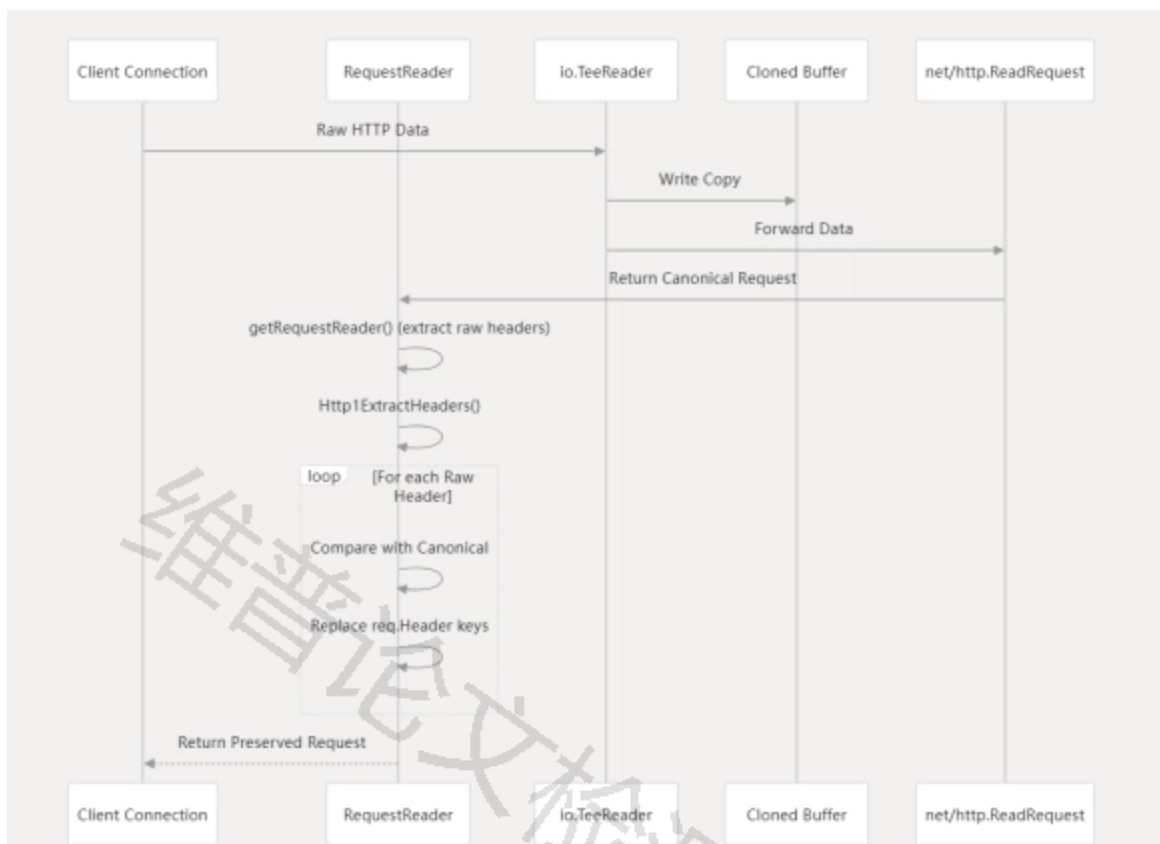
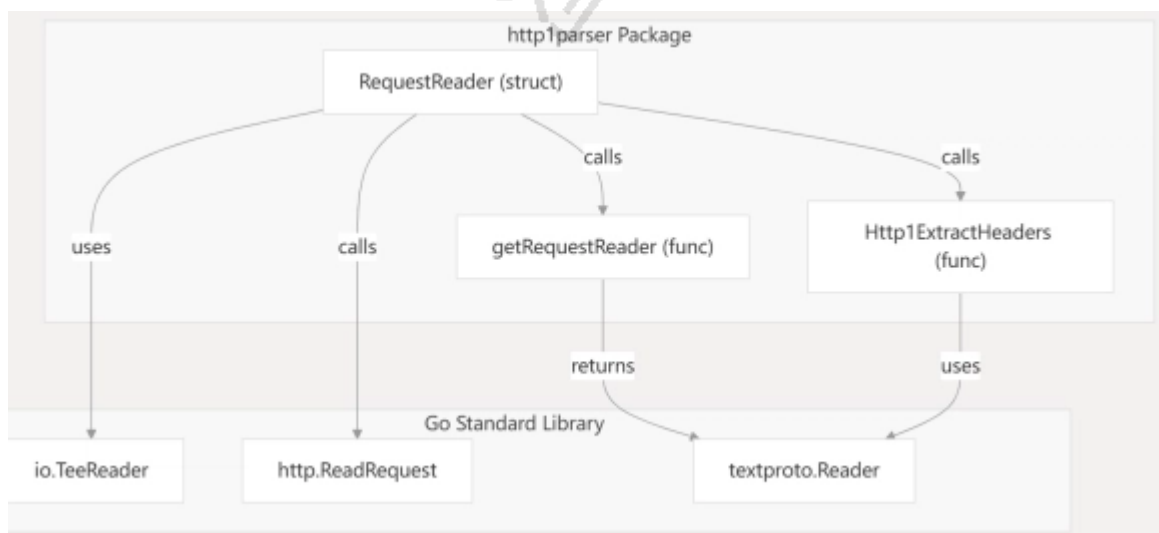


图6-1 http解析器处理流程



6.2.3 访问控制测试

访问控制测试覆盖 mproxy/actions_test.go。该文件验证客户端 IP 归一化、客户端 IP 提取优先级、HTTP 请求按来源 IP 拦截、CONNECT 请求按来源 IP 拒绝以及封禁规则命中后关闭活跃连接等逻辑。访问控制既作用于普通 HTTP 请求，也作用于 CONNECT 隧道入口，因此单元测试需要同时覆盖两类入口，才能支撑第 5 章中请求责任链和 CONNECT 责任链的实现说明。

6.2.4 MinIO 存储测试

MinIO 存储测试覆盖 myminio/mino_upload_test.go、myminio/minio_temp_cleanup_test.go、myminio

/storage_api_test.go 和 myminio/minio_download_api_test.go。测试内容包括 Body 流式捕获、空 Body 捕获、大文件 Body 捕获、未知长度临时文件清理、下载链接构造、缺少 Key、空 Key、对象不存在和有效 Key 下载等场景。由于 MITM 抓包正文不直接放入 WebSocket 消息，而是通过对象 Key 关联到 MinIO 中的请求体或响应体对象，因此该部分测试直接关系到抓包记录能否被前端下载和复核[5]。

6.2.5 goroutine 泄露检查测试

goroutine 泄露检查测试集中在 test/gotest/goroutine_test.go。测试代码通过 go.uber.org/goleak 验证 BuildBodyReader、Pipe 写入、MinIO 上传协程和 Close 流程是否能够正常退出。已知长度路径使用 Content-Length 走流式上传，未知长度路径使用 contentType=-1 走临时文件路径，二者分别对应代理运行中的 KnownSize 和 Chunked 场景。

该部分测试的判定重点不是单次吞吐，而是读写完成后后台协程是否回收、Close 是否超时、并发上传是否发生死锁。对于代理系统而言，goroutine 泄露会在长时间运行中逐步放大资源消耗，最终表现为连接堆积、内存增长或响应延迟增加。因此泄露检查是压力测试之外的稳定性验证手段。

6.3 功能测试设计

6.3.1 HTTP 代理功能测试

HTTP 代理功能测试将客户端代理地址配置为 127.0.0.1:8080，并向 http://127.0.0.1:9011/test/upload 和 /test/download 发起请求。上传请求使用 5MB 文件作为请求体，下载请求从测试后端读取同一文件。测试时应观察三个结果：客户端是否收到 200 响应；前端连接页面是否出现 HTTP 连接记录；概览页中的实时上行、下行和累计流量是否随请求变化。

6.3.6 前端页面功能测试

前端页面功能测试从登录页开始，输入控制服务地址和密钥后进入控制台。概览页应显示实时上传下载曲线、累计流量和连接摘要；连接页应支持筛选、排序、展开父子连接和关闭连接；日志页应支持等级筛选、分类筛选和正则搜索；MITM 页面应支持搜索、排序、展开头部和下载正文；路由、安全策略、存储配置和高级设置页面保存后应能重新读取并保持表单状态一致。

表 6-3 主要功能测试用例

编号	测试项	输入或操作	预期结果
FT-01	HTTP 代理	通过代理访问 9011 上传与下载接口	响应正常返回，连接、日志和流量统计更新
FT-02	HTTPS 隧道	MITM 关闭时访问 9012 HTTPS 接口	CONNECT 隧道建立，只统计加密字节流
FT-03	MITM 抓包	MITM 开启后发起 GET、POST 和 5MB 上传下载	生成明文交换记录，正文对象可下载
FT-04	Chunked 传输	上传不设置 Content-Length，下载访问 /test	请求和响应均可完整转发，

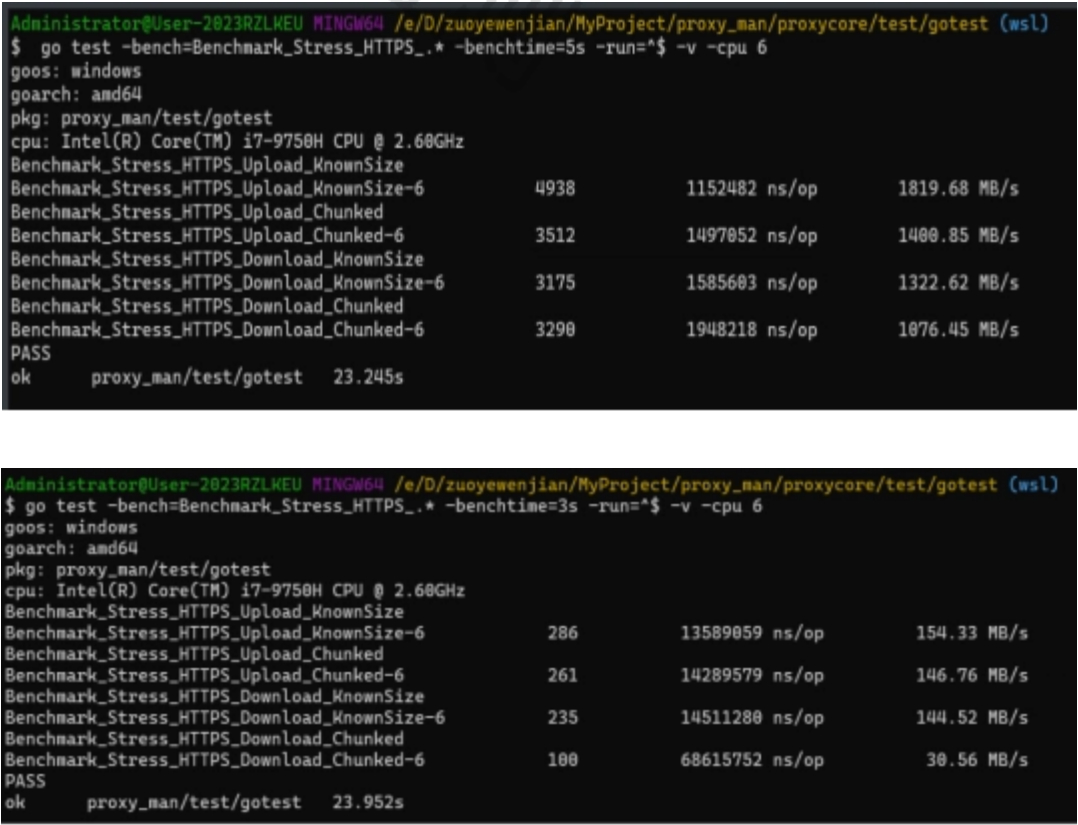
		/download/chunked	无长度不一致错误
FT-05	路由规则	新增代理节点和规则后发起命中请求	日志出现匹配结果，未命中时回退直连
FT-06	访问控制	新增目标规则或来源 IP 封禁	HTTP 返回 403 或 CONNECT 被拒绝，拦截日志更新
FT-07	前端页面	登录并依次访问概览、连接、日志、MITM 和配置页	页面状态与后端推送或配置接口一致

6.4 性能测试设计

6.4.1 5MB 文件上传下载测试

我们分别选用 5MB和100MB作为测试文件，覆盖 HTTP 上传、HTTPS 上传、HTTP 下载和 HTTPS 下载。在 MITM 未开启时，HTTPS 上传下载主要验证 CONNECT 隧道的双向字节复制能力；在 MITM 开启时，HTTPS 上传下载还会经过 TLS 终止、HTTP 解析、请求响应快照生成和对象存储捕获等流程。通过两组结果对比，可以观察 MITM 所带来的额外开销。

图6-1 压力测试



测试结果表明最大传输速度可达1GB/s 以上，开启中间人劫持后传输速度依然能达到100MB/s以上。

6.4.2 高并发压力测试

高并发压力测试采用 Go benchmark 的 b.RunParallel 产生并发请求，并使用 b.SetBytes 将每轮处理字节数

计入吞吐统计。test/gotest/fetch_test.go 中的 stressClient 和 stressClientHTTPS 均配置了代理地址、连接池、空闲连接数量和超时时间，其中 HTTPS 客户端在测试环境中跳过证书验证，避免自签名证书影响压测过程。

表 6-4 性能测试矩阵

维度	测试内容	关注指标
协议维度	HTTP 与 HTTPS	HTTPS 隧道或 MITM 带来的额外 CPU 开销
方向维度	上传与下载	上行、下行路径中的流量统计和对对象捕获开销
传输维度	KnownSize 与 Chunked	已知长度流式上传与未知长度临时文件路径差异
MITM 维度	MITM 关闭与开启	明文解析、证书、抓包记录和 MinIO 捕获开销
并发维度	-cpu 6 与 -cpu 8	物理核心口径和更高调度压力下的稳定性

benchmark 输出中需要重点记录 ns/op、B/op、allocs/op 和吞吐相关指标。对于代理系统，单纯追求最高吞吐并不足够，还需要观察错误率、连接关闭是否及时、前端连接列表是否持续刷新、日志是否出现大量异常以及 pprof 中是否存在异常热点。

图6-2 压力测试

```
administrator@User-2023RZLKEU MINGW64 /e/D/zuoyewenjian/MyProject/proxy_man/proxycore/test/gotest (wsl)
$ go test -bench=.*(Chunked|KnownSize)$* -benchtime=3s -run=*$ -v -cpu 6
goos: windows
goarch: amd64
pkg: proxy_man/test/gotest
cpu: Intel(R) Core(TM) i7-9750H CPU @ 2.60GHz
BenchmarkStressHTTPUploadKnownSize
BenchmarkStressHTTPUploadKnownSize-6                266           14614761 ns/op           143.58 MB/s
BenchmarkStressHTTPUploadChunked
BenchmarkStressHTTPUploadChunked-6                    181           17214268 ns/op           121.83 MB/s
BenchmarkStressHTTPSUploadKnownSize
BenchmarkStressHTTPSUploadKnownSize-6                 234           16148818 ns/op           129.93 MB/s
BenchmarkStressHTTPSUploadChunked
BenchmarkStressHTTPSUploadChunked-6                   193           18888806 ns/op           111.55 MB/s
BenchmarkStressHTTPDownloadKnownSize
BenchmarkStressHTTPDownloadKnownSize-6                259           12896618 ns/op           162.61 MB/s
BenchmarkStressHTTPDownloadChunked
BenchmarkStressHTTPDownloadChunked-6                  180           35818789 ns/op            59.89 MB/s
BenchmarkStressHTTPSDownloadKnownSize
BenchmarkStressHTTPSDownloadKnownSize-6               272           48813315 ns/op            52.41 MB/s
BenchmarkStressHTTPSDownloadChunked
BenchmarkStressHTTPSDownloadChunked-6                 180           235949641 ns/op             8.89 MB/s
BenchmarkStressHTTPMITMUploadKnownSize
BenchmarkStressHTTPMITMUploadKnownSize-6              230           27813387 ns/op            75.48 MB/s
BenchmarkStressHTTPMITMUploadChunked
BenchmarkStressHTTPMITMUploadChunked-6                 64           89618941 ns/op            23.48 MB/s
BenchmarkStressHTTPMITMDownloadKnownSize
BenchmarkStressHTTPMITMDownloadKnownSize-6            180           42868751 ns/op            48.92 MB/s
BenchmarkStressHTTPMITMDownloadChunked
BenchmarkStressHTTPMITMDownloadChunked-6              180           58464689 ns/op            41.56 MB/s
BenchmarkUploadChunked
BenchmarkUploadChunked-6                               8234          1975425 ns/op
PASS
ok      proxy_man/test/gotest    112.149s
```

使用最大开销https+MITM中间人在大流量并发下依然能保持在100M/s左右的传输速度。

6.4.3 pprof 性能分析

pprof 分析在大流量压测期间进行。堆内存采集用于观察 5MB 请求体和响应体是否被完整读入内存，以及 MinIO 流式上传和 Chunked 临时文件策略是否降低了堆内存增长风险。CPU 采集用于观察热点是否集中在 io.Copy、TLS 握手、HTTP 解析、JSON 序列化、对象存储上传或 WebSocket 推送等位置。

我们在该系统设计中的预期资源特征是：KnownSize 传输应主要表现为流式读取和流式上传，内存占用不应随文件大小按完整文件倍数增长；Chunked 传输由于需要处理未知长度数据，可能增加临时文件写入和分块编码处理开销；MITM 场景相比普通隧道会增加 TLS、HTTP 解析和抓包元数据构造开销。pprof 采集结果可用于验证这些预期是否成立，并为后续优化提供依据。

图6-3 cpu性能测试

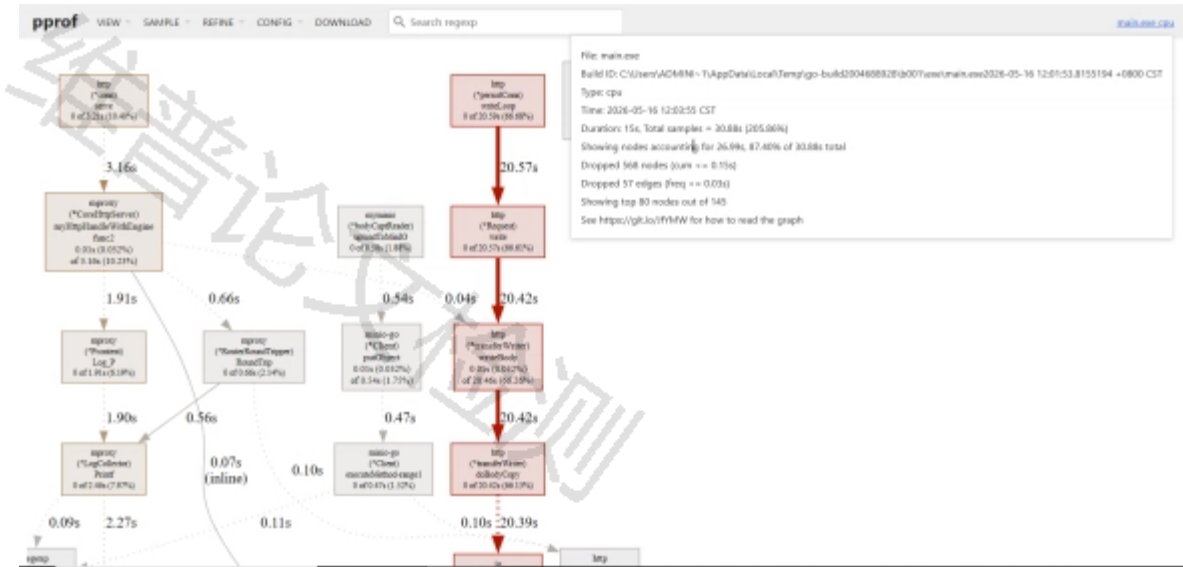


图6-4 内存占用测试



图6-5 压力测试

```
Administrator@User-2023HZLMEU RINGMO4 /e/D/zuoyuwenjian/MyProject/proxy_man/proxycore/test/gotest (wsl)
$ go test -bench=Benchmark_Stress_HTTP_Upload_KnownSize -benchtime=12s -run=$ -v -cpu 6
goos: windows
goarch: amd64
pkg: proxy_man/test/gotest
cpu: Intel(R) Core(TM) i7-9750H CPU @ 2.60GHz
Benchmark_Stress_HTTP_Upload_KnownSize
Benchmark_Stress_HTTP_Upload_KnownSize-6          2166          35124269 ns/op          59.71 MB/s
PASS
```

```
ok proxy_man/test/gotest 76.925s
```

在https + MITM劫持的情况下每秒200次并发的情况下非chunked模式，cpu占用33%左右，内存稳定占用5M左右。

图6-6 cpu性能测试

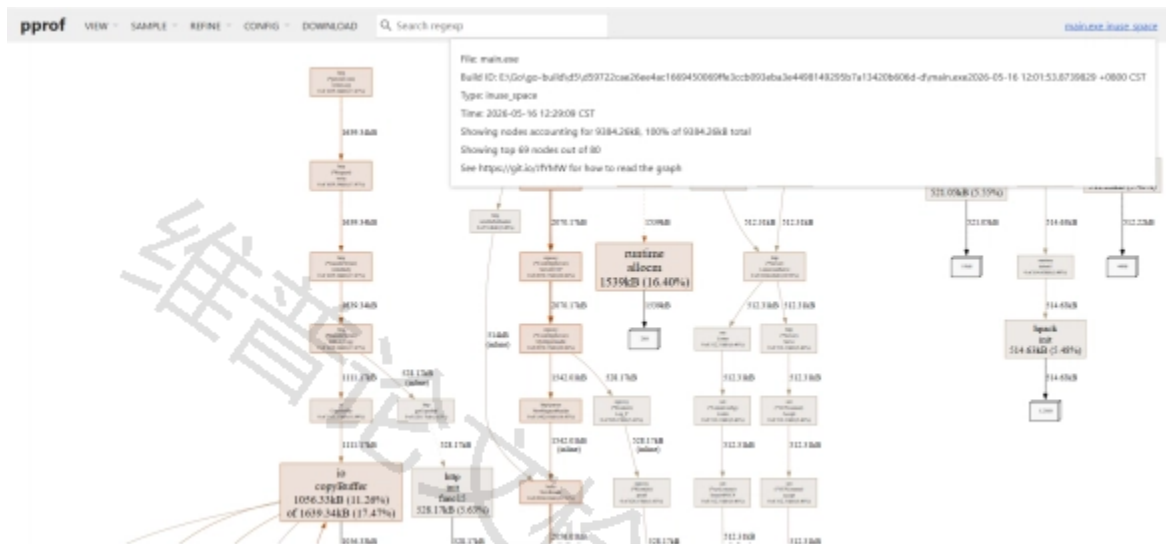


图6-7 内存开销测试

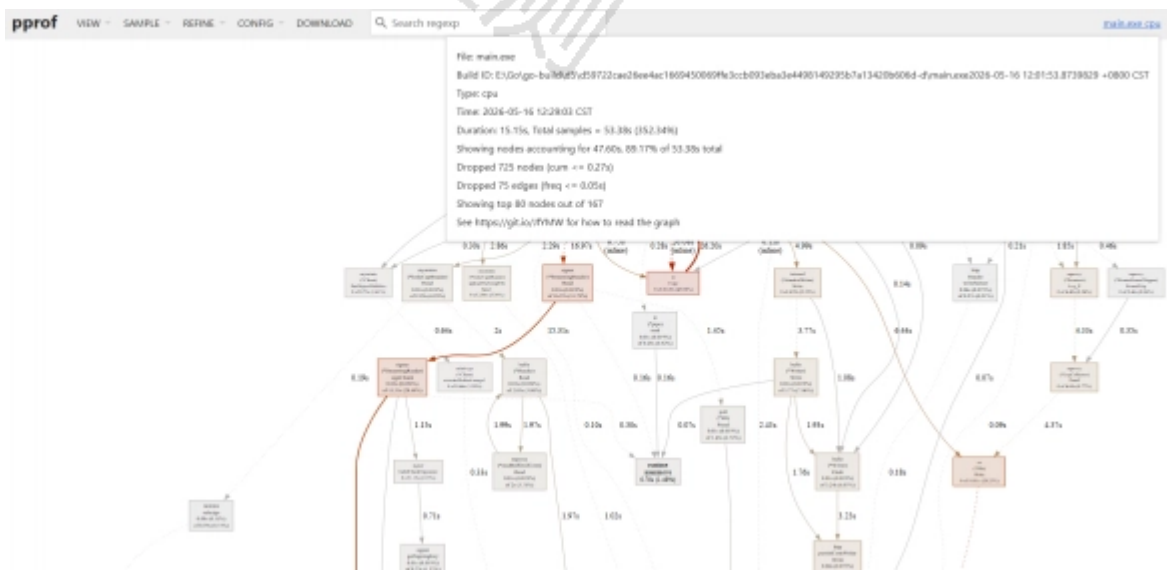


图6-8 压力测试

```
Administrator@User-2023R2LKEU MINGW64 /e/D/zuoyewenjian/MyProject/proxy_man/proxycore/test/gotest (wsl)
$ go test -bench=Benchmark_Stress_HTTP_MITM_Upload_Chunked -benchtime=12s -run=$ -v -cpu 6
goos: windows
goarch: amd64
pkg: proxy_man/test/gotest
cpu: Intel(R) Core(TM) i7-9750H CPU @ 2.60GHz
Benchmark_Stress_HTTP_MITM_Upload_Chunked
Benchmark_Stress_HTTP_MITM_Upload_Chunked-6      1585      16578401 ns/op      126.50 MB/s
PASS
ok proxy_man/test/gotest 27.339s
```


在https + MITM劫持的情况下每秒70次并发的情况下chunked模式传输100M大文件，cpu占用80%左右，内存稳定占用10M左右。原因是cpu主要用于处理每次拷贝，从内核内存到用户内存再从用户内存到内核socket缓冲区内存，但是内存开销极低。

6.5 测试结果分析

6.5.1 功能完整性分析

从测试覆盖关系看，本章功能测试能够对应需求分析中的主要功能项。HTTP 代理和 HTTPS 隧道测试用于验证基础转发能力；MITM 抓包测试用于验证授权调试场景下的明文解析、请求响应记录和正文下载

6.5.2 性能与资源占用分析

性能分析应以未开启 MITM 的 HTTP/HTTPS 结果作为基线，再与开启 MITM 后的结果进行比较。未开启 MITM 时，HTTP 请求可以快速响应，HTTPS 请求主要经过 CONNECT 隧道和双向复制。开启 MITM 后，系统会额外执行 TLS 握手、HTTP 请求解析、抓包快照生成和 MinIO 捕获等步骤，会存在额外的cpu开销。但是内存开销始终稳定，因为只需要小缓冲区就能完成大文件上传。

6.5.3 问题与改进分析

测试设计也暴露出系统后续可以继续完善的方向。第一，控制服务当前主要依赖 token 认证，适合本地实验和课程演示，但如果面向多用户场景，还需要增加账号体系、权限分级、操作审计和 HTTPS 控制通道。第二，系统目前主要围绕 HTTP/1.1 进行解析和测试，后续可以扩展到 HTTP/2、HTTP/3 和 gRPC 场景。第三，运行日志、连接历史和抓包记录当前主要保存在内存及对象存储中，后续可以增加数据库索引，以支持长期检索、审计和报表统计。

7 总结与展望

本章在前文需求分析、总体设计、详细实现和系统测试的基础上，对基于 Go 与 Vue 的可视化网络代理管理系统进行归纳总结。全文围绕网络代理核心、可视化控制台、实时监控、MITM 抓包、规则路由、访问控制和对象存储等内容展开，既说明了系统为什么需要这些功能，也说明了这些功能如何在代码中落地，并通过测试章节对主要功能链路和资源占用风险进行了验证。

总体来看，我们在该系统设计中完成了从代理接入、请求转发、运行观测、策略配置到抓包留存的闭环设计。后端以 Go 的网络编程能力和并发控制能力为基础，承担 HTTP/HTTPS 代理、CONNECT 隧道、MITM 明文解析、路由选择、访问控制、流量统计和对象存储等职责；前端以 Vue 3 生态为基础，为用户提供概览、连接、日志、抓包、路由、安全、用户监控、存储和高级配置等页面。系统实现过程体现了毕业设计从问题提出、方案设计、编码实现到测试分析的完整工程实践路径。

7.1 工作总结

7.1.1 系统完成情况

本课题完成了一套可运行的可视化网络代理管理系统。后端部分以 CoreHttpServer 为核心入口，能够区分普通 HTTP 请求和 CONNECT 请求，并根据配置进入普通转发、隧道透传、HTTP-MITM 或 HTTPS-MITM 等不同处理分支。普通 HTTP 请求通过请求责任链完成访问控制、路由注入、流量统计和请求体捕获，再经由 RoundTrip 转发至目标服务。

测试方面，系统围绕 HTTP 解析器、配置管理、访问控制、MinIO 存储、goroutine 泄露检查、HTTP/HTTPS 转发、MITM 抓包、Chunked 传输、压力基准和 pprof 分析建立了验证思路。第 6 章已经说明，测试并不只关注客户端是否收到成功响应，还会同时检查连接状态、流量统计、日志记录、抓包对象和资源释放情况。通过这种测试组织方式，可以较全面地验证系统在开发调试和课程演示场景下的可用性与稳定性。

7.2 系统特色

7.2.1 代理核心与控制台一体化

我们在该系统设计中的第一个特色是代理核心与 Web 控制台的一体化设计。后端程序不仅提供 HTTP 代理端口和 HTTPS 代理端口，还通过控制服务提供静态前端资源、WebSocket 实时通信、REST 配置接口、统计查询接口和对象下载接口。前端构建产物嵌入 Go 二进制后，系统可以以单进程方式运行，从而降低部署和演示成本。

这种一体化结构比较适合毕业设计场景。传统做法可能需要分别启动代理服务、前端开发服务、配置服务和对象下载服务，演示链路相对较长；我们在该系统设计中把代理运行状态和管理页面放在同一个控制入口下，用户只需访问控制服务地址，就可以观察流量、查看连接、调整配置和下载抓包正文。对于本地调试、课堂演示和小规模实验环境，这种方式更容易复现，也更便于说明系统整体架构。

一体化设计并不意味着模块边界混乱。代码中代理核心、配置管理、路由、访问控制、MinIO、WebSocket Hub 和前端页面仍然保持相对独立。控制服务只是把这些能力组织到统一入口中，实际处理逻辑仍分布在各自模块内。这样的结构既保证了使用上的集中，也保留了后续扩展和维护的可能性。

7.2.2 实时监控与可操作连接表

我们在该系统设计中的第二个特色是将实时监控与连接管理结合起来。后端通过 WebSocket 按主题推送 traffic、connections、log_batch、mitm_exchange_batch、intercept_log_batch 和 user_traffic 等消息，前端根据页面需要订阅对应主题。用户不仅可以看到总上传、总下载、实时速率和最近流量曲线，还可以在连接页查看当前活动连接、父子连接关系、协议、Host、URL、来源 IP 和流量大小。

连接表不是只读列表，而是具备一定操作能力。前端可以关闭全部连接、关闭指定连接，也可以在用户监控页按 IP 断开连接或封禁来源 IP。后端连接记录中保存关闭回调，前端操作通过 WebSocket 发送到后端后，能够影响正在运行的代理连接。这样，系统从单纯的“观测工具”进一步扩展为“可干预的管理工具”。

为了适应高频数据场景，系统在前后端都做了减压处理。后端日志、抓包记录和拦截记录采用批量推送，连接和用户流量按固定周期刷新；前端连接、历史连接、日志和 MITM 列表使用虚拟滚动减少 DOM 数量，并对历史数据设置保留上限。该设计使控制台在大量连接和日志产生时仍能保持较好的可用性。

7.2.3 流式抓包与对象存储结合

我们在该系统设计中的第三个特色是将 MITM 抓包与对象存储结合。MITM 抓包通常容易遇到一个问题：如果请求体和响应体较大，直接把正文放入内存或通过 WebSocket 推送给前端，会明显增加代理进程和浏览器端压力。我们在该系统设计中在实时消息中只推送请求和响应的元数据，正文内容则通过 MinIO 保存为对象。前端页面在需要查看正文时，再通过下载接口获取对应内容，而不是让 WebSocket 直接承载大体积数据。

7.3 不足之处

7.3.2 协议覆盖仍可扩展

当前系统主要围绕 HTTP/1.1 代理、HTTPS CONNECT 隧道、HTTP-MITM 和 HTTPS-MITM 展开设计。对于大多数课程演示、接口调试和基础代理场景，这一范围已经能够覆盖常见需求；但现代网络应用中还存在 HTTP/2、HTTP/3、gRPC、WebSocket 子协议和更多长连接通信形式，系统在这些方向上仍有扩展空间。

HTTP/2 和 HTTP/3 与 HTTP/1.1 在连接复用、头部压缩、流控制和传输层协议上存在明显差异。若后续需要支持这些协议，不能简单复用当前的 HTTP/1.1 解析器，而需要在连接层、请求流模型和抓包展示方式上重新设计。尤其是在 MITM 场景中，多路复用请求与父子连接关系的展示方式也需要调整。

WebSocket 当前主要作为长连接流量转发和控制服务通信协议使用，系统不会解析具体业务消息。后续如果面向更深入的调试场景，可以考虑在授权条件下增加 WebSocket 帧级观测、消息大小统计、文本帧查看和二进制帧下载等能力。但该方向需要注意隐私边界和性能开销，不能影响普通代理转发的稳定性。

7.3.3 数据持久化能力仍可增强

当前系统的运行日志、历史连接、用户流量聚合和 MITM 交换记录主要保存在内存中，请求体和响应体正文保存在对象存储中。这样的设计适合实时观察和短期调试，但对于长期运行、历史检索和审计分析仍然不够充分。系统重启后，内存中的连接历史和日志索引会丢失，用户难以按时间、Host、状态码或来源 IP 进行跨会话查询。

后续可以引入关系型数据库或轻量级嵌入式数据库保存抓包索引、连接摘要、日志摘要、配置变更历史和用户操作记录。对象存储继续保存大体积正文，数据库保存元数据和检索字段，两者通过对象 Key 关联。这样既不需要把正文塞入数据库，也可以为前端提供分页查询、条件筛选、审计导出和统计报表等能力。

数据持久化增强后，还需要配套清理策略。抓包正文和日志长期积累会占用磁盘或对象存储空间，系统应支持按时间、大小、Host、状态码或用户设置的保留周期自动清理。只有同时考虑写入、检索和清理，持久化能力才能真正适合长期使用。

7.4 展望

7.4.1 多用户与权限管理

后续系统可以引入多用户管理能力，将用户划分为管理员、调试人员和只读观察者等角色。管理员可以修改代理端口、路由规则、访问控制和存储配置；调试人员可以查看连接、日志和 MITM 抓包记录；只读观察者只能查看概览和部分统计数据。通过角色权限划分，可以降低误操作风险，也便于在团队调试场景中共享同一个代理控制台。

在多用户基础上，还可以增加操作审计。系统应记录用户登录、配置修改、MITM 开关变更、连接关闭、IP 封禁、正文下载和规则导入导出等操作。审计记录可用于问题回溯，也能让高风险操作具备责任边界。对于毕业设计后续完善而言，多用户与权限管理是从单机实验工具走向团队化管理平台的重要一步。

7.4.2 更丰富的规则体系

当前路由和访问控制规则主要支持域名后缀、域名关键词和 IP 匹配。后续可以扩展为更丰富的规则体系，例如基于请求方法、路径、Header、状态码、内容类型、用户 IP、时间窗口和流量大小进行匹配。规则动作也可以从“直连、走二级代理、拦截”扩展为“限速、重写 Header、记录审计、仅抓包不拦截”等更细粒度行为。

规则管理界面也可以继续增强。后续可支持规则分组、优先级拖拽、启停开关、批量导入导出、命中次数统计和规则测试工具。用户在保存规则前，可以输入目标 URL 或 Host 进行模拟匹配，提前看到会命中哪条规则以及对

应动作。这样可以降低规则配置错误带来的代理中断风险。通过本次毕业设计，课题较好地综合运用了网络协议、后端并发、前端工程、对象存储和软件测试等知识，达到了工程实践与论文研究相结合的目标。

致谢

感谢指导教师在选题、需求分析、系统设计和论文写作过程中的指导。

感谢同学、朋友或测试使用者在功能验证和问题反馈中的帮助。

感谢开源社区提供 Go、Vue、MinIO、Chart.js、Pinia、Vite 等技术生态。

参考文献

- [1] Fielding R, Nottingham M, Reschke J. HTTP Semantics[S/OL]. RFC 9110, 2022[2026-05-16]. <https://www.rfc-editor.org/rfc/rfc9110>.
- [2] Fette I, Melnikov A. The WebSocket Protocol[S/OL]. RFC 6455, 2011[2026-05-16]. <https://www.rfc-editor.org/rfc/rfc6455>.
- [3] The Go Authors. Package net/http[EB/OL]. (2026-05-07)[2026-05-16]. <https://pkg.go.dev/net/http>.
- [4] Vue.js Team. Introduction | Vue.js[EB/OL]. [2026-05-16]. <https://vuejs.org/guide/introduction.html>.
- [5] MinIO. MinIO Go Client SDK for Amazon S3 Compatible Cloud Storage[EB/OL]. [2026-05-16]. <https://minio-go.min.io/>.
- [6] The Go Authors. Package sync[EB/OL]. (2026-04-07)[2026-05-16]. <https://go.dev/pkg/sync/>.
- [7] Vite Team. Getting Started | Vite[EB/OL]. [2026-05-16]. <https://vite.dev/guide/>.
- [8] Vue Router Team. Getting Started | Vue Router[EB/OL]. [2026-05-16]. <https://router.vuejs.org/guide/>.
- [9] Pinia Team. State | Pinia[EB/OL]. [2026-05-16]. <https://pinia.vuejs.org/core-concepts/state.html>.
- [10] Chart.js Contributors. Getting Started | Chart.js[EB/OL]. (2025-10-13)[2026-05-16]. <https://www.chartjs.org/docs/latest/getting-started/>.
- [11] TanStack. Vue Virtual | TanStack Virtual Vue Docs[EB/OL]. [2026-05-16]. <https://tanstack.com/virtual/latest/docs/framework/vue/vue-virtual>.
- [12] The Go Authors. Package embed[EB/OL]. [2026-05-16]. <https://go.dev/pkg/embed/>.
- [13] Gamma E, Helm R, Johnson R, et al. Design Patterns: Elements of Reusable Object-Oriented Software[M]. Boston:Addison-Wesley, 1994.

附录

本附录对系统运行配置、控制接口、WebSocket 消息格式以及测试构建命令进行集中整理。附录内容依据 proxycore/config.json、mproxy/config.go、proxysocket/proxy.go、proxysocket/hub.go、proxyui/src/stores/websocket.js、Makefile 和测试目录材料整理，用于支撑正文中的系统复现、接口说明和测试验证。

附录 A 配置文件示例

系统运行配置采用 JSON 文件保存，后端启动时通过环境变量 PROXY_MAN_CONFIG_PATH 指定配置文件位置。配

置管理器会在启动阶段读取该文件；当文件不存在时，系统根据默认配置创建新文件并写入磁盘。该设计使代理端口、MITM 开关、路由规则、访问控制规则、来源 IP 封禁规则和 MinIO 连接信息能够脱离代码独立维护。

表 A-1 配置文件主要字段说明

字段	类型	示例或默认值	说明
Port	number	8080	HTTP 代理监听端口，客户端或测试工具通过该端口接入普通 HTTP 代理请求。
HTTPSPort	number	8443	HTTPS 代理监听端口，服务端固定 HTTP/1.1 以保证 CONNECT 隧道可以使用 Hi jack。
Verbose	boolean	true	控制运行日志输出详细程度，便于在开发调试和论文测试阶段观察代理处理过程。
KeepAcceptEncoding	boolean	false	控制是否保留客户端 Accept-Encoding 头部，关闭时有利于代理读取明文响应体。
PreventParseHeader	boolean	false	控制是否跳过部分头部解析逻辑，主要用于特殊调试场景。
KeepDestHeaders	boolean	true	控制转发响应时是否保留目标服务器响应头。
ConnectMaintain	boolean	false/true	控制 CONNECT 隧道是否维持长连接，影响隧道透传和连接生命周期。
MitmEnabled	boolean	false/true	MITM 总开关。开启后系统可在授权环境中解析 HTTPS 明文请求与响应。
HttpMitmNoTunnel	boolean	false/true	控制非 CONNECT 的 HTTP 请求是否进入 HTTP-MITM 处理路径。
PublicIPs	array	["117.72.191.85"]	公网 IP 或域名列表，用于代理自环检测和证书监听地址补充。
			路由总开关。关闭时系统默认使

RouteEnable	boolean	true	用 Direct 直连出站。
ProxyNodes	array	clash 节点	二级 HTTP 代理节点列表，每个节点包含 Name 和 URL。
Routes	array	DomainKeyword 规则	路由规则列表，支持 DomainSuffix、DomainKeyword 和 IP 三类匹配。
AccessEnable	boolean	true	访问控制总开关。关闭时目标拦截和来源 IP 封禁不生效。
AccessRules	array	example.com	目标访问控制规则，可按域名后缀、域名关键词或 IP 拦截。
UserBlockRules	array	192.168.1.10	来源客户端 IP 封禁规则，保存后会关闭已命中的活跃连接。
MinioConfig	object	bodydata	对象存储配置，用于保存 MITM 请求体和响应体。

配置文件示例

```
{
  "Port": 8080,
  "HTTPSPort": 8443,
  "Verbose": true,
  "KeepAcceptEncoding": false,
  "PreventParseHeader": false,
  "KeepDestHeaders": true,
  "ConnectMaintain": true,
  "MitmEnabled": true,
  "HttpMitmNoTunnel": true,
  "PublicIPs": ["117.72.191.85"],
  "RouteEnable": true,
  "ProxyNodes": [
    {"Name": "clash", "URL": "http://127.0.0.1:7892"}
  ],
  "Routes": [
    {"Id": 1, "Type": "DomainKeyword", "Value": "(.*|^)google\\.com$", "Action": "clash", "Enable":
```

```
true, "Remarks": "示例路由"}
],
"AccessEnable": true,
"AccessRules": [
{"Id": 1, "Type": "DomainSuffix", "Value": "example.com", "Enable": true, "Remarks": "示例拦截"}
],
"UserBlockRules": [
{"Id": 1, "Value": "192.168.1.10", "Enable": true, "Remarks": "示例来源 IP 封禁"}
],
"MinioConfig": {
"endpoint": "minio:9000",
"publicEndpoint": "",
"accessKeyId": "root",
"secretAccessKey": "12345678",
"useSSL": false,
"bucket": "bodydata",
"enabled": true
}
}
```

Docker 部署时，配置路径通常设置为 /data/proxy_config/config.json。若旧配置仍将 MinIO 内网地址写为 127.0.0.1:9000，配置管理器会在该 Docker 路径下自动迁移为 minio:9000，避免容器内服务无法访问对象存储。PublicEndpoint 为空时，浏览器下载 Body 数据会通过控制服务转发；若配置了公网 Endpoint，则可生成面向浏览器的预签名下载地址。

附录 B 主要接口说明

控制服务默认监听 8000 端口，既提供前端静态资源，也提供 REST 配置接口、统计接口、对象存储下载接口和 WebSocket 入口。当前实现中 WebSocket 入口通过 token 参数校验密钥，REST 接口主要面向本地实验和受信任网络环境；若系统用于多人或公网环境，应在后续版本中补充统一认证、权限分级和 HTTPS 控制通道。

表 B-1 REST 与 WebSocket 入口说明

接口	方法	参数或请求体	功能说明
/api/config	GET	无	读取当前 ServerConfig 配置，前端路由、安全、存储和高级设置页面均依赖该接口初始化表单。
			保存配置到磁盘。保存成功后触

/api/config	POST	ServerConfig JSON	发路由热重载、访问控制热重载，并关闭命中来源 IP 封禁的活跃连接。
/api/stats	GET	无	返回进程级统计数据，目前包含 interceptCount，用于概览页和安全策略页展示累计拦截次数。
/api/storage/download	GET	key; 可选 proxy=1	根据 MinIO 对象 Key 生成下载信息；proxy=1 时由控制服务直接转发对象内容。
/start	WebSocket	token	WebSocket 实时推送入口。 token 与服务端 Secret 不一致时返回 401，连接成功后客户端发送 subscribe 动作订阅主题。
/	GET	前端路由路径	返回嵌入的 Vue 前端静态资源；不存在的无扩展名路径回退到 index.html，由 Vue Router 接管。

表 B-2 WebSocket 客户端动作说明

动作	主要字段	后端处理逻辑	典型页面
subscribe	topics、logLevel	更新当前连接订阅主题，决定是否推送 traffic、connections、logs、mitm_detail、intercept_logs 和 user_traffic。	概览、连接、日志、MITM、安全策略、用户监控
closeAllConnections	无	遍历活动连接并调用 onClose 回调，同时标记连接关闭。	概览页、连接页
closeConnection	id	关闭指定 Session 及其子连接，随后从连接表中移除。	连接页
closeUserConnections	ip	规范化来源 IP，关闭该 IP 对应的全部活跃连接。	用户监控页
cleanOfflineUsers	无	根据当前连接表保留在线用户，清理离线用户流量记录并立即广播。	用户监控页

```
{  
  "code": 0,  
  "message": "success",  
  "data": {  
    "downloadUrl": "http://127.0.0.1:8000/api/storage/download?key=mitm-data%2F2026-02-04%2F10086%2F2Freq&proxy=1",  
    "expiresAt": "2026-05-16T12:00:00+08:00",  
    "filename": "10086_req.bin",  
    "size": 5242880  
  }  
}
```

附录 C WebSocket 消息格式

订阅消息示例

```
{
  "action": "subscribe",
  "topics": ["traffic", "connections", "logs", "mitm_detail", "intercept_logs", "user_traffic"],
  "logLevel": "INFO"
}
```

表 C-1 服务端推送消息说明

消息类型	推送周期或触发方式	主要字段	前端用途
traffic	每 1 秒	data.up、data.down、data.totalUp、data.totalDown	概览页绘制最近 60 秒上传下载速率曲线，并展示累计流量。
		id、parentId、host、	连接页和概览页展示活动连

connections	每 500 毫秒	method、url、remote、 clientId、protocol、 status、up、down	接、父子关系、协议类型和 流量变化。
log_batch	每 300 毫秒或满 200 条	level、session、message、 category、time	日志页按等级、分类和关键 字筛选后展示运行日志。
mitm_exchange_batch	每 500 毫秒或满 100 条	id、sessionId、 parentId、request、 response、duration、error	MITM 页面展示明文请求响 应、头部、耗时、对象 Key 和错误信息。
intercept_log_batch	每 500 毫秒或满 100 条	data、interceptCount	安全策略页展示实时拦截记 录，并同步累计拦截次数。
user_traffic	每 1.5 秒	ip、online、totalUp、 totalDown、hosts	用户监控页按来源 IP 和 Host 维度展示用户流量，并 支持断开或封禁用户。

traffic 消息示例

```
{
  "type": "traffic",
  "data": {"up": 10240, "down": 20480, "totalUp": 1048576, "totalDown": 2097152}
}
```

connections 消息数据项示例

```
{
  "id": 10086,
  "parentId": 10080,
  "host": "example.com",
  "method": "GET",
  "url": "https://example.com/api/list",
  "remote": "127.0.0.1:53000",
  "clientId": "127.0.0.1",
  "protocol": "HTTPS-MITM",
  "status": "Active",
  "up": 512,
  "down": 4096
}
```

MITM 交换消息要点

MITM 交换记录由 `HttpExchange` 结构生成。`request` 字段保存 `method`、`url`、`host`、`header`、`sumSize`、`bodyKey`、`bodySize`、`bodyUploaded`、`contentType` 和 `bodyError`；`response` 字段保存 `statusCode`、`status`、`header`、`sumSize`、`bodyKey`、`bodySize`、`bodyUploaded`、`contentType` 和 `bodyError`。前端收到后会将嵌套结构扁平化，便于表格排序、筛选、展开头部和下载请求体或响应体。

user_traffic 消息示例

```
{
  "type": "user_traffic",
  "data": [
    {
      "ip": "127.0.0.1",
      "online": true,
      "totalUp": 1024,
      "totalDown": 4096,
      "hosts": [
        {
          "host": "example.com",
          "up": 1024,
          "down": 4096
        }
      ]
    }
  ]
}
```

附录 D 测试与构建命令

本附录整理系统开发、构建、测试和性能分析阶段常用命令。前端命令在 `proxyui` 目录执行，后端命令在 `proxycore` 目录执行，压力测试命令在 `proxycore/test/gotest` 目录执行。运行后端前必须设置 `PROXY_MAN_CONFIG_PATH`，使配置文件路径明确可控。

表 D-1 开发与构建命令

场景	命令	说明
前端开发服务	<code>cd proxyui; npm run dev</code>	启动 Vite 开发服务器，适合单独调试前端页面。
前端生产构建	<code>cd proxyui; npm run build</code>	生成 <code>dist</code> 静态资源，后续可复制到后端嵌入目录。
后端本地运行	<code>cd proxycore; 设置 PROXY_MAN_CONFIG_PATH; go run .</code>	启动控制服务、HTTP 代理服务和 HTTPS 代理服务。
一体化 Linux 构建	<code>cd proxycore; make build</code>	先构建前端，再复制 <code>dist</code> 到 <code>proxysocket/dist</code> ，最后编译 Linux 可执行文件。
后端单元测试	<code>cd proxycore; go test ./...</code>	运行配置、HTTP 解析器、访问控制、MinIO 和日志等单元测试。
测试后端服务	<code>cd proxycore; go run test /backend_server.go</code>	启动上传、下载和 HTTPS 测试目标服务。

PowerShell 本地启动示例

```
cd proxycore
```

```
$env:PROXY_MAN_CONFIG_PATH = ".\config.json"
```

```
go run .
```

Go benchmark 压力测试示例

```
cd proxycore/test/gotest
```

```
go test -bench=Benchmark_Stress_HTTP.* -benchtime=10s -run=$ -v -cpu 6
```

```
go test -bench=Benchmark_Stress_HTTPS.* -benchtime=10s -run=$ -v -cpu 6
```

```
go test -bench=".*(Chunked|KnownSize)$" -benchtime=10s -run=$ -v -cpu 6
```

```
go test -bench=Benchmark_Stress_HTTP_Upload_KnownSize -benchtime=20s -run=$ -v -cpu 8
```

上述 benchmark 覆盖 HTTP 上传、HTTP 下载、HTTPS 上传、HTTPS 下载、KnownSize 和 Chunked 等场景。测试前应先启动测试后端、代理核心和 MinIO，并根据 MITM 开关分组记录结果。论文正文中的第 6 章可结合 benchmark 输出的 ns/op、B/op、allocs/op、吞吐量和错误日志进行分析。

goroutine 泄露检查示例

```
cd proxycore/test/gotest
```

```
go test -v -run=TestConcurrent_RealNetwork_KnownLength -timeout 10m
```

```
go test -v -run=TestConcurrent_RealNetwork_Chunked -timeout 10m
```

pprof 采集示例

```
go tool pprof -http=:8081 http://localhost:6060/debug/pprof/heap
```

```
go tool pprof -http=:8083 "http://localhost:6060/debug/pprof/profile?seconds=15"
```

pprof 命令需要在代理进程开启 net/http/pprof 调试服务后执行，并应在压力测试运行期间采集。堆内存数据用于观察大文件上传下载时是否出现异常内存增长，CPU 数据用于观察热点是否集中在 io.Copy、TLS 握手、HTTP 解析、JSON 序列化、对象存储上传或 WebSocket 推送等环节。

附录内容到此结束。

须知：

- 报告编号系送检论文检测报告在本系统中的唯一编号。
- 本报告结合AI大模型自动生成，结果仅供参考。
- 报告支持中、英文内容检测。



微信公众号

维普论文检测